

所属类别	2025 年“华数杯”全国大学生数学建模竞赛	参赛编号
本科组		CM2502608

基于粒子群算法与统计检验的光照光谱优化设计及睡眠质量影响分析

摘要

发光二极管（LED）是一种性能极好的新型环保光源，不仅能够提供视觉照明，还可以影响人体的生理节律系统。于是思考如何在满足照明需求的同时，优化 LED 光源的光谱特性，以达到有益于人类的生理节律调节效应。因此本文通过分阶段、分场景的设计，结合统计检验、优化算法等工具，实现了从光谱建模到睡眠影响分析的完整逻辑闭环。

针对问题一，首先通过给定的 SPD 数据计算标准 CIE XYZ。基于 CIE1931 标准，求得的色品坐标为 (0.3840445003795555, 0.3767800565662821)；基于 CIE 1964 标准，计算的色品坐标为 (0.3875379505804716, 0.37337820391292564)。接着通过黑轨迹的 Chebyshev 法计算得到的相关色温 (CCT) 为 390.6483K，进一步通过 McCamy 近似公式法进行验证，二者差值仅为 0.4704K。接着通过查阅资料独缺黑体曲线色品坐标数据，得到色偏差 (Duv) 的值为 -0.001024。对于颜色还原参数，计算较为复杂，基于色度空间转换原理，建立计算模型，最终求得保真度指数 (Rf) 值为 91.79，色域指数 (Rg) 的值为 106.08。对于生理节律效应参数，我们首先计算黑视素与明视觉之间的比值，再通过褪黑素日光照度比 (mel-DER) 与该比值呈线性的关系，求得该 SPD 的褪黑素日光照度比 (mel-DER) 的值为 0.640847。

针对问题二，根据不同模式的光谱要求建立不同的优化模型。(1) 对于日光照明模式，目标是模拟正午日光 (相关色温 CCT=6500K)，构建以“合成光谱的保真度指数 (Rf) 最大化”为目标函数的优化模型。(2) 对于夜间助眠模式，目标是在相关色温 CCT 处于 2500K~3500K 区间内，构建以“合成光谱的视黑素日光照度比 (mel-DER) 最小化”为目标函数的优化模型。采用粒子群算法对这个非线性优化问题进行求解：(1) 日间模式最优权重：蓝光: 0.1552 绿光: 0.1766 深红光: 0.2335 暖白光: 0.0149 冷白光: 0.4199 (2) 夜间模式最优权重：蓝光: 0 绿光: 0.1136 红光: 0 暖白光: 0.8864 冷白光: 0。将该合成光谱带入问题一的参数计算模型中得到五大参数特征值。

针对问题三，与问题 2 类似，仍然是寻找最佳的权重组合来合成满足特定需求的光谱。需要找到 LED 通道所分配的权重随时间段变化。选用切比雪夫范数输出 15 组误差中的最大值。本题的目标就是最小化 15 组误差中的最大值，使所有组误差都达标，采用粒子群算法迭代输出最优的组合权重。成功绘制了拟合曲线以及特定时刻的拟合对比图。

针对问题四，首先计算求得 33 个样本的指标值。其次对六大指标进行正态性检验和方差齐性检验，(1) 对于总睡眠时间、深睡眠比例、REM 睡眠比例、夜间醒来次数使用 **重复测量方差分析方法**。(2) 对于睡眠效率、入睡潜伏期这两个指标采用 **非参数的 Friedman 检验方法**。分析统计检验结果，只有深度睡眠阶段占比这一指标对应的 P 值 = **0.0492** (小于 0.05)，对于该指标三种环境有显著性差异。进一步进行描述性统计分析得到，“优质光照”与“黑暗环境”相比，黑暗环境在提高 N3 睡眠阶段占比方面表现更优。进一步通过两两检验，分析发现，仅从以上统计结果分析，难以有力证明“优化光照”能在多个方面显著提高睡眠质量。但是可以间接说明“优质光照”在睡眠调控方面具有较大的提升潜力。

关键词：CIE XYZ 非线性规划 粒子群算法 重复测量方差分析方法 Friedman 检验方法

一、问题重述

1.1 问题背景

科学研究表明，光照除了能够为我们提供照明，同时也能够通过视网膜来影响人体的生理节律系统。近年来发光二极管（LED）能够通过调节色温和光谱特性，满足不同的照明需求，因此我们思考将二者结合起来。希望在满足照明这一基本需求的同时，我们希望利用好 LED，优化其光源的光谱特性以此实现有益的生理节律效应，为人类幸福事业添砖加瓦。因此在本文将建立数学模型以此探究 LED 光谱特性，以及研究在不同的场景下的最佳策略，并且通过实验检验策略的以下是具体的问题的提出。

1.2 问题提出

问题一：需要根据题目给出的一组 LED 光源 SPD 数据（波长及其对应的光谱规律），参考相关专业文献，给出五个参数：相关色温（CCT）、距离普朗克轨迹的距离（Duv）、保真度指数（Rf）和色域指数（Rg）、褪黑素日照度比（mel-DEI）的标准化计算方法和数学模型，并且计算出附件所给的 SPD 数据对应的五个特征参数值。

类别	指标	解释
颜色特性参数	相关色温（CCT）	描述光源的“颜色外观”
	距离普朗克轨迹的距离（Duv）	
颜色还原参数	保真度指数（Rf）	评估光源与标准光源相比，还原物体真实色彩的能力
	色域指数（Rg）	
生理节律效应参数	褪黑素日照度比（mel-DEI）	量化光照对人体生理节律的影响程度

问题二：需要根据题目所给出的五个独立 LED 通道（深红光、绿光、蓝光、暖白光、冷白光）的 SPD 数据，寻找最佳的权重组合来合成满足“日间照明模式”和“夜间助眠模式”这两个场景下的特定需求的光谱。然后分别给出最优通道权重组合以及这两个合成光谱的五个关键参数（CCT、Duv、Rf、Rg、mel-DEI）的值

问题三：需要根据问题二中的五通道 LED 数据，设计一个控制策略，使其合成的光谱能够在全天范围内模拟附件给出的太阳光谱数据。使其合成光能够与太阳光有相同的节律效果。进一步，选取三个代表性时间点的合成光谱，绘制合成光谱与目标太阳光谱的对比图，进行效果验证。

问题四：在上述问题求解完成后，在理论上优化得出了特定模式的光谱。进一步，需要验证该优化光谱的现实适应性。首先根据题目提供的原始数据，为每一次睡眠记录计算出客观睡眠评估指标，其次使用统计检验方法，分析三种光照环境（A、B、C）对各项睡眠指标的影响是否存在显著性差异，最后给出准确的结论，我们设计的“优化光照（环境 A）”相比于“普通光照（环境 B）”和“黑暗环境（环境 C）”是否对睡眠质量具有改善作用？

二、问题分析

2.1 问题一

针对问题一，(1) 对于第一类颜色特性参数：相关色温 (CCT) 和距离普朗克轨迹的距离 (D_{uv})，我们先将题目给出的 SPD 数据转换到标准的 CIE XYZ 色彩空间。其次我们再通过算法来寻找其在普朗克轨迹上最近的点来确定两个指标值。(2) 对于第二类颜色还原参数：保真度指数 (Rf) 和色域指数 (Rg)，将题目已给的 SPD 数据照射在一系列标准化的“测试色样”上，再通过查阅的文献得到的公式比较两者在色样上引起的差异进而计算得出指标值。(3) 对于第三类生理节律效应参数：褪黑素日光照度比 (mel-DER)，通过查阅文献可以得到计算公式。我们发现，以上指标的计算，不同学者从多重角度进行研究，得出不同的计算方法，因此我们将考虑多种方法，多标准参照，以此进行交叉验证，进一步判读模型的稳定性。

2.2 问题二

针对问题二，根据 LED 的任意光谱合成可以采用线性叠加的原理，可以将多个 LED 的光谱分布加权求和，得到满足特定需求的光谱。包含两个场景：(1) 日间照明模式；(2) 夜间助眠模式。我们根据不同模式的光谱要求建立不同的优化模型。对于日间照明模式，目标是模拟正午日光 (CCT=6500K) 的条件下，使得合成光谱的保真度指数 (Rf) 尽可能高。核心约束为色温 (CCT) 范围控制，色域指数 (Rg) 范围控制。对于夜间助眠模式，目标是在合成光谱的相关色温在 (2500K, 3500K) 在该区间时，使得合成光谱的褪黑素日光照度比 (mel-DER) 尽可能低。核心约束为保真度指数下限约束，以及相关色温范围控制。粒子群算法的高效收敛性及对约束的灵活适应性，与本问题中光谱合成的多约束、多参数耦合特性高度匹配，能可靠求解两个场景的最优 LED 组合权重，之后再将合成光谱代入问题一建立的计算模型，即可输出对应的五个核心参数，完成场景化光谱的全流程分析。

2.3 问题三

针对问题三，要求结合问题二中给出的五通道 LED，设计一个控制策略，使其合成的光谱能够在全天范围模拟给定的太阳光谱数据，使其具有相似的节律效果。我们要使每个时间段合成的光谱都可以模拟太阳光谱的节律效果。与问题 2 类似，仍然是寻找最佳的权重组合来合成满足特定需求的光谱。控制策略意思就是，需要找到 LED 通道所分配的权重随时间段变化。我们首先计算太阳光谱的 CCT，其次得到合成光谱的 CCT 矩阵。通过标准化转换后，我们可以得到 15 组 CCT 数据。此时可以得到 15 组待求解数据和目标数据 (太阳光谱的 CCT)，我们选用切比雪夫范数输出 15 组误差中的最大值。本题的目标就是最小化 15 组误差中的最大值，使所有组误差都达标。求解与问题二一样，采用粒子群算法迭代输出最优的组合权重。最终绘制拟合曲线以及特定时刻的拟合对比图。

2.4 问题四

针对问题四，首先分析题目给出的实验数据，一共有 33 个样本（11 位测试者，每位测试者在 3 种环境下的数据），根据六大指标的定义式，转换为可以由编码数据计算的数学计算式，最终得到计算求得指标值。其次对六大指标进行正态性检验和方差齐性检验，若在三种环境均满足正态性，且方差齐性，则采用重复测量方差分析；若至少在一个环境不满足正态性，则采用非参数的 Friedman 检验。根据这个检验结果，分别判断每个指标在三种环境照射后是否具有显著性的差别。为了进一步判断“优质光照”是否会对人类睡眠质量改善。我们将采用两两假设检验（A 与 B，A 与 C）进一步判断“优化光照”是否能带来睡眠质量的有益的改善。最终我们可以根据统计检验结果对“优质光照”是否能够对人类睡眠质量得到改善得到初步结论。

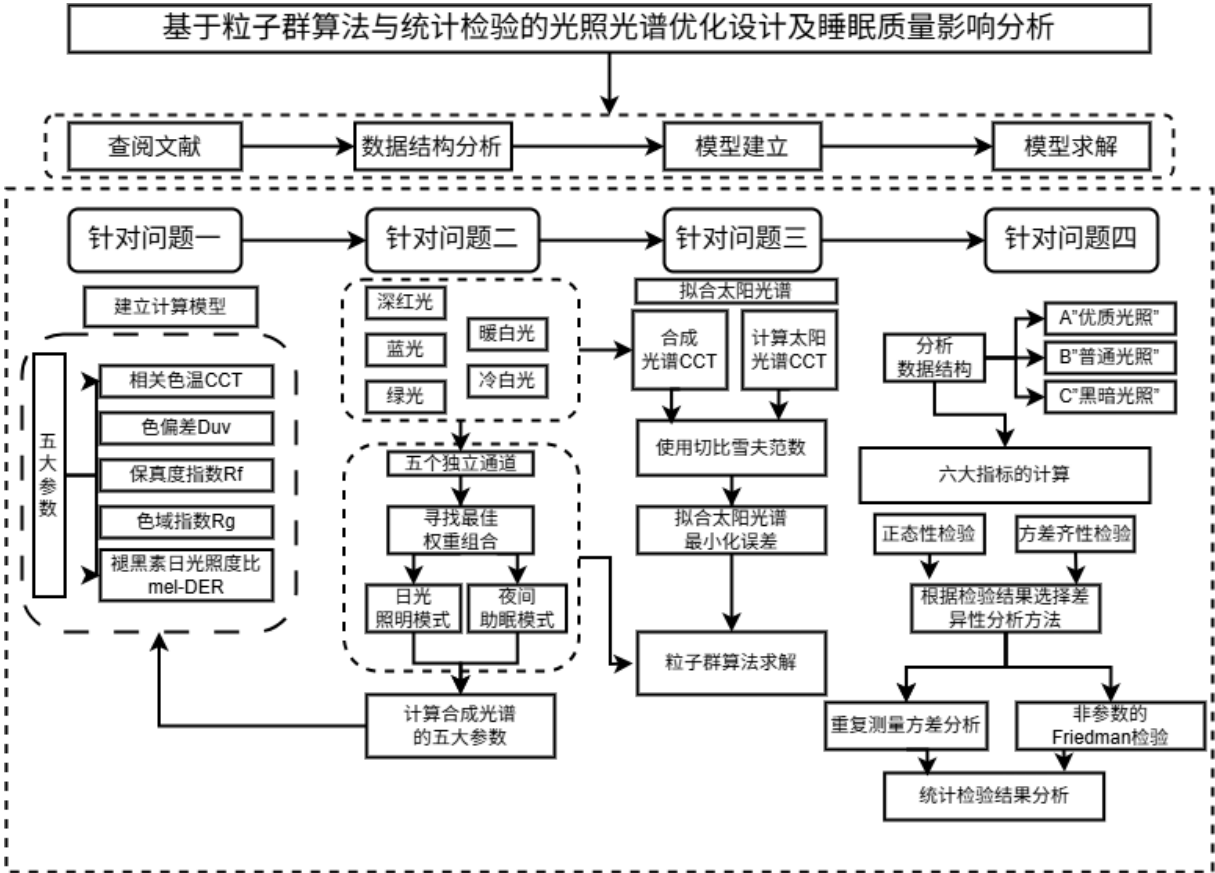


图 1 本文的框架图

三、模型假设和符号说明

3.1 模型基本假设

- 1.假设不同 LED 通道的光谱混合满足线性叠加原理，忽略荧光转换和非线性效应。
- 2.假设各 LED 通道间无光耦合效应。
- 3.假设 CCT、Rf、Rg 等参数的计算模型在离散波长下保持足够精度。
- 4.假设日间模式的目标色温和夜间模式的色温区间是基于场景需求的合理设定，且约束条件（如 Rf 下限）符合实际应用标准。
5. 假设五通道 LED 的光源特性不随时间、温度等因素漂移。
6. 假设测试者在三种环境下的睡眠数据相互独立(除个体差异外无其他混淆变量)，且数据记录无系统性误差。
7. 假设 33 个样本具有代表性，能反映目标人群在三种光照环境下的睡眠特征。

3.2 符号说明

符号	说明
$I(\lambda)$	光谱数据（波长 λ 对应的 SPD）
$\bar{x}(\lambda), \bar{y}(\lambda), \bar{z}(\lambda)$	CIE 标准观察者的三刺激值函数
(x, y)	色品坐标
$d\lambda$	波长间隔（1nm）
K	归化系数
R_w, G_w, B_w	光源白场的 RGB 值
J	明度
h	色调角
M	视彩度
ΔE	色貌差
ΔE_{ave}	平均色貌差
$\Phi_e(\lambda)$	待测光源光谱曲线
$V(\lambda)$	明视觉光谱曲线
$S_{mel}(\lambda)$	黑视素敏感曲线
α	各通道光谱的叠加比例

以上符号是本文建立数学模型所涉及的符号，后文模型中涉及到符号也会详细说明

四、问题一的模型建立与求解

光源的光谱功率分布（SPD）是描述其物理特性最基础的数据，记录了光源在各个波长上的能量分布。题目给定了一组 LED 光源的 SPD 数据，拟合后如下图：波长与其对应的光谱功率。下面我们通过查阅文献以及相关的资料，参考不同学者的计算方法，提出以下指标的标准化方法和计算模型，最终计算得出该组数据的五个特征参数值。

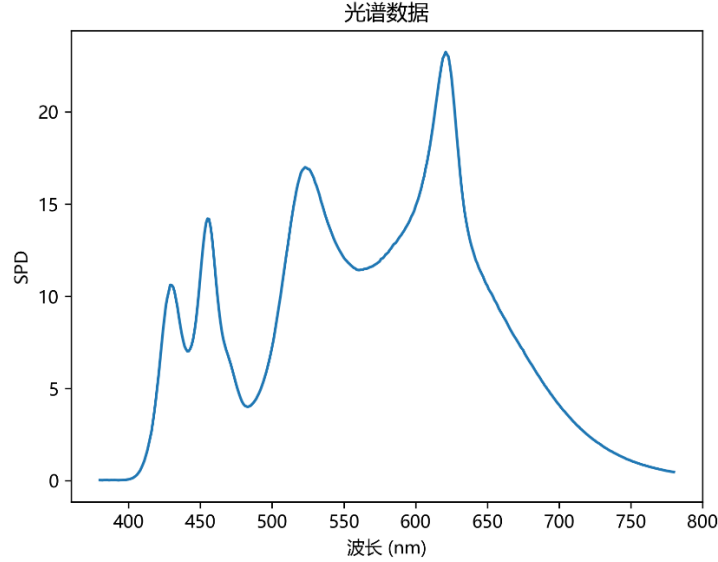


图 2 光谱波长-SPD

4.1 颜色特性参数

4.1.1 相关色温（CCT）

Step1: 计算标准 CIE XYZ

CIE 的计算是基于 CIE 标准观察者函数，这些函数描述了人眼对不同波长光的感知效应。我们选择了两种主流的标准：CIE 1931 标准和 CIE 1964 标准。

CIE 1931 标准	2° 视场，仅覆盖人眼中央凹区域的锥体细胞
CIE 1964 标准	10° 视场，适合手掌大小的物体时的颜色感知

因此我们得出结论：CIE 1931 标准 适合观察一小块物体的颜色感知，CIE 1964 覆盖更广的视场，更适合 LED 灯的相关色温测量。如下为观察者函数公式。

$$\begin{cases} X = k \int_{\lambda} I(\lambda) \cdot \bar{x}(\lambda) \cdot d\lambda \\ Y = k \int_{\lambda} I(\lambda) \cdot \bar{y}(\lambda) \cdot d\lambda \\ Z = k \int_{\lambda} I(\lambda) \cdot \bar{z}(\lambda) \cdot d\lambda \end{cases} \quad (1)$$

$$k = \frac{100}{\sum_{\lambda} I(\lambda) \bar{y}(\lambda) \Delta \lambda} \quad (2)$$

其中， $I(\lambda)$ 为光谱数据（波长 λ 对应的 SPD）； $\bar{x}(\lambda), \bar{y}(\lambda), \bar{z}(\lambda)$ 为 CIE 标准观察者的三刺激函数； $d\lambda$ 为波长间隔：1nm； K 为归化系数。

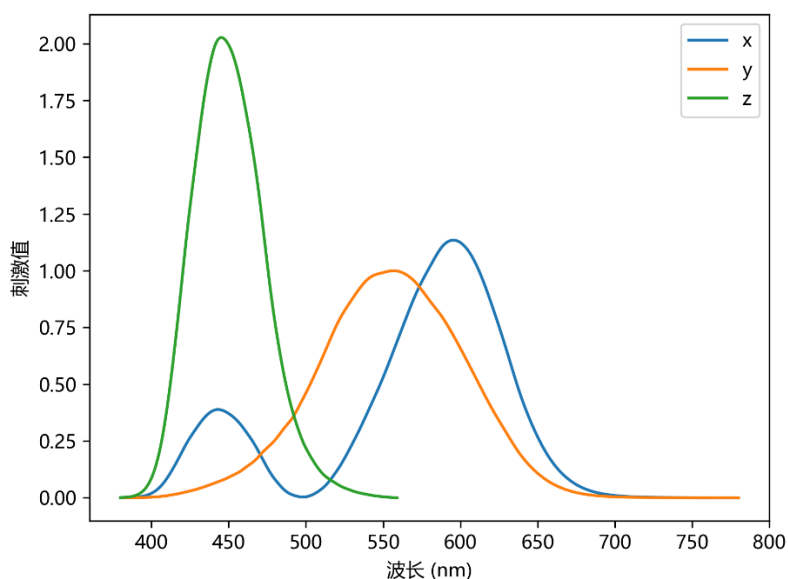


图 3 CIE 标准观察者函数

题目给的是间隔为 $1nm$ 的数据，所以我们下面对于每个波长 λ ，带入下面公式计算出来 XYZ 坐标。

$$X = k \sum_{\lambda} I(\lambda) \cdot \bar{x}(\lambda) \cdot \Delta\lambda \quad (3)$$

$$Y = k \sum_{\lambda} I(\lambda) \cdot \bar{y}(\lambda) \cdot \Delta\lambda \quad (4)$$

$$Z = k \sum_{\lambda} I(\lambda) \cdot \bar{z}(\lambda) \cdot \Delta\lambda \quad (5)$$

经过以上，便计算得到 XYZ 坐标，最终我们需要得到 (x,y) 色品坐标，因此需要进行归一化处理。

$$\begin{cases} x = \frac{X}{X+Y+Z} \\ y = \frac{Y}{X+Y+Z} \end{cases} \quad (6)$$

最终我们基于两种标准计算出来的色品坐标为：

标准	色品坐标
CIE 1931 标准	(0.3840445003795555, 0.3767800565662821)
CIE 1964 标准	(0.3875379505804716, 0.37337820391292564)

Step2:计算相关色温 (CCT) ^[1]

相关色温指的是与光源色坐标靠最近的合体色坐标点对应的黑体温度。以上已经求得色品坐标，通过查阅文献，发现共有 5 种计算相关色温的方法。包括：三角垂足插值法、黑体轨迹的 Chebyshev 法、模拟黑体轨迹弧线法、McCamy 近似公式法、牛顿迭代法。我们选择黑体轨迹的 Chebyshev 法进行计算，选择 McCamy 近似公式法进行验证。

(1) 黑体轨迹的 Chebyshev 法进行计算：

CIE 1931 的 (x,y) 坐标要转成 CIE 1960 (u,v) 坐标，

$$\begin{cases} u = \frac{4x}{-2x+12y+3} \\ v = \frac{6y}{-2x+12y+3} \end{cases} \quad (7)$$

设温度为 T 的黑体色品坐标为 $[u(T), v(T)]$ ，其中 $c(T)$ 代表 $u(T)$ 或者 $v(T)$ 。
满足：

$$\min \left[\max_{T_i < T < T_f} \left| c(T) - \frac{P(T)}{Q(T)} \right| \right] \quad (8)$$

根据 Remes 的方法，当 $1000K \leq T \leq 15000K$ 时，

$$\begin{cases} \bar{u}(T) = \frac{0.860117757 + 1.54118254 \times 10^{-4}T + 1.28641212 \times 10^{-7}T^2}{1 + 8.42420235 \times 10^{-4}T + 7.08145163 \times 10^{-7}T^2} \\ \bar{v}(T) = \frac{0.317398726 + 4.22806245 \times 10^{-5}T + 4.20481691 \times 10^{-8}T^2}{1 - 2.89741816 \times 10^{-5}T + 1.61456053 \times 10^{-7}T^2} \end{cases} \quad (9)$$

根据周等温线垂直于黑体轨迹，得

$$\frac{u(T_c) - u_c}{v(T_c) - v_c} = -\frac{dv(T_c)}{du(T_c)} \quad (10)$$

其中 (u_c, v_c) 是待求相关色温 T_c 的色品坐标，只需解下列方程便可以得到相关色温 T_c 。

$$\frac{du(T_c)}{dT_c} |u(T_c) - u_c| + \frac{dv(T_c)}{dT_c} |v(T_c) - v_c| = 0 \quad (11)$$

采用黑体轨道的 Chebyshev 法求得的相关色温为 3903.6483K

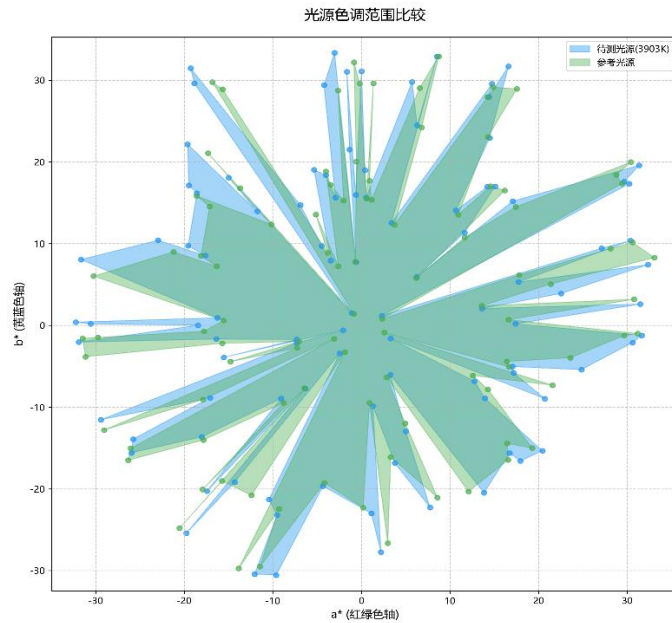


图 4 待测光源与参考光源色调范围比较

(2) 采用较为简单的 McCamy 近似公式法计算，对结果进行验证。

$$T = -437n^3 + 3601n^2 - 6861n + 5514.31 \quad (12)$$

其中 $n = \frac{x-0.3320}{y-0.1858}$ ，得出来相关色温为 3903.1779 K。

使用黑体轨道的 Chebyshev 法进行计算相关色温为 3903.6483K；使用 McCamy 近似公式法计算的相关色温为 3903.1779 K。两种方法计算的相关色温的差值仅为 0.4704K。验证了结果的准确性。

4.1.2 距离普朗克轨迹的距离 (Duv)

通过阅读文献，得到普朗克轨迹的距离公式为：

$$Duv = [sign(v_s - v_t)] \sqrt{(u_t - u_s)^2 + (v_t - v_s)^2} \quad (13)$$

同样需要将 CIE 1931 色品坐标 (x, y) 转换为 CIE 1960 色品坐标 (u, v) ：

$$\begin{cases} u = \frac{4x}{-2x+12y+3} \\ v = \frac{6y}{-2x+12y+3} \end{cases} \quad (14)$$

黑体轨迹上对待测光源点最近的点的坐标。查阅相关资料，我们需要读取黑体曲线色品坐标数据，数据源自书籍《LED 件选型与评价》康玉柱（著）。

查阅文献，发现只有 $|Duv| \leq 0.054$ 才具有实际意义^[2]。我们最终求得的距离普朗克轨迹的距离 (Duv) 为 -0.001024，因此其具有实际研究意义。

4.2 颜色还原参数

4.2.1 保真度指数 (Rf) ^[3]

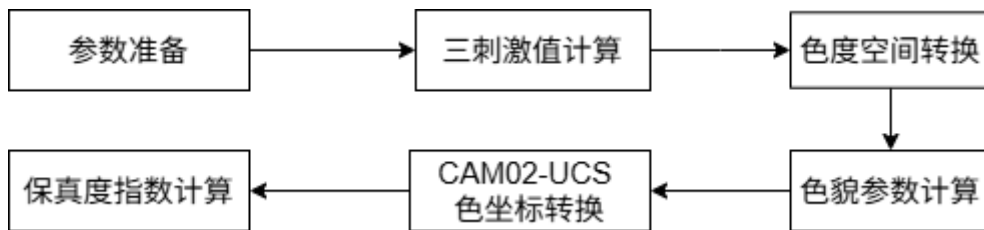


图 5 保真度指数计算流程

(1) 参数准备：待测光源的相关色温小于 4000k，选择参照光源为同色温黑体光谱

(2) 三刺激值计算：分别计算待测光源 (X_t, Y_t, Z_t) 和参考光源 (X_r, Y_r, Z_r) 的 CIE 1964 10° 视角三刺激值，以及 99 个标准色样在两种光源下的三刺激值 (X_{wt}, Y_{wt}, Z_{wt}) 和 (X_{wr}, Y_{wr}, Z_{wr}) 。计算式上述 (1) 中已给出。

(3) 色度空间转换：

Step1 XYZ→CAT02 RGB：通过转换矩阵 M_{cat02} 实现：

$$\begin{pmatrix} R \\ G \\ B \end{pmatrix} = M_{cat02} \begin{pmatrix} X \\ Y \\ Z \end{pmatrix} \quad (15)$$

Step2 Von Kries 色适应：将色样在待测/参照光源下的 RGB 值适应到光源白场：

$$R_c = 100 \cdot \frac{R}{R_w}, G_c = 100 \cdot \frac{G}{G_w}, B_c = 100 \cdot \frac{B}{B_w} \quad (16)$$

其中， R_w ， G_w ， B_w 为光源白场的 RGB 值。

Step3 通过逆矩阵 M_{cat02}^{-1} 转回 XYZ

Step4 Hunt-Pointer-Estevéz 变换：将 XYZ 转换为 R' G' B'

$$\begin{pmatrix} R' \\ G' \\ B' \end{pmatrix} = M_{HPE} \begin{pmatrix} X \\ Y \\ Z \end{pmatrix} \quad (17)$$

Step5 亮度适应：对 R' G' B' 进行非线性变换

$$R_a = \frac{400 \cdot \left(\frac{f_L \cdot R'}{100} \right)^{0.42}}{27.13 + \left(\frac{f_L \cdot R'}{100} \right)^{0.42}} + 0.1 \quad (18)$$

G'、B'类似， $f_L = 0.7937$ 。

(4) 色貌参数计算^[3]

无彩色相应 A ： $A = 1.0003 \cdot (2R_a + G_a + 0.05B_a - 0.305)$

明度 J ： $J = 100 \cdot \left(\frac{A}{A_w} \right)^{0.69 \times 1.9272}$ (A_w 为光源白场的 A)

色调角 h ： $h = \arctan 2(b, a) \times \frac{180}{\pi}$

视彩度 M ：

$$\begin{cases} 1. \text{色调系数 } e: e = 0.25 \cdot (\cos(h_{\text{rad}} + 2) + 3.8) (h_{\text{rad}} \text{ 为弧度的色调角}) \\ 2. \text{彩度因子 } t: t = \frac{50000 / 13 \times 1.0003 \times e \times \sqrt{a_w^2 + b_w^2}}{R_a + G_a + (21/20)B_a} \\ 3. \text{彩度 } C: C = t^{0.9} \cdot \sqrt{J/100} \cdot (1.64 - 0.29^{0.2})^{0.73} \\ 4. \text{视彩度 } M: M = C \cdot 0.7937^{0.25} \end{cases}$$

(5) CAM02-UCS 色坐标转换

将明度、视彩度、色调角转换为均匀色空间坐标 (J' , a' , b')：

$$\begin{cases} \text{明度 } J': J' = \frac{(1+100 \times 0.007) \times J}{1+0.007 \times J} \\ \text{视彩度 } M': M' = \frac{1}{0.0228} \cdot \ln(1+0.0228 \times M) \\ \text{色品坐标: } a' = M' \cdot \cos(h_{\text{rad}}), b' = M' \cdot \sin(h_{\text{rad}}) \end{cases}$$

(6) 色彩度保真度指数 (Rf) 计算

色貌差 ΔE : 每个样品在待测与参照下的 (J', a', b') 欧式距离:

$$\Delta E = \sqrt{(J'_t - J'_r)^2 + (a'_t - a'_r)^2 + (b'_t - b'_r)^2} \quad (19)$$

平均色貌差:

$$\Delta E_{\text{ave}} = \frac{1}{99} \sum \Delta E \quad (20)$$

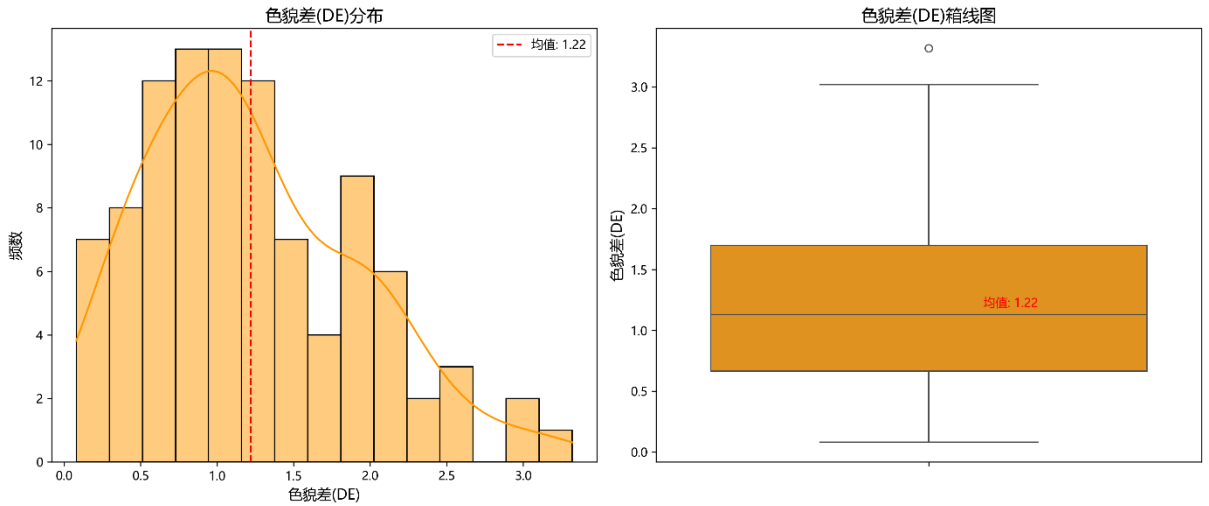


图 6 色貌差的描述性统计量

最终计算得到色彩保真度指数为 91.79。

4.2.2 色域指数 (Rg) [3]

色域指数是 TM-30 用于相对色域面积的度量, 它近似表示所有色调的平均色饱和度变化。计算 Rg 首先需要将 99 个 CIE 标准观察者分布到 16 个色调角箱中, 以便计算稳定面积。将每个箱中所有样品的 (a', b') 坐标取平均值, 形成测试条件和参考条件下各一个 16 边形的顶点。Rg 是两个多边形面积之商 (测试面积除以参考面积), 再乘以 100。

(1) 色调角按 Bin 划分

将 99 个色样按参照光源下的色调角 h_r 分为 16 个 Bin (每个 Bin 覆盖 22.5°), 即第 j 个 Bin 对应色调角范围 $(22.5(j-1), 22.5j]$ 。

(2) 各 Bin 的平均坐标计算

对每个 Bin, 计算待测光源与参照光源下色样的 a', b' 平均值:

$$\text{待测光源: } (\bar{a}'_{t,j}, \bar{b}'_{t,j}) = \frac{1}{N_j} \sum_{i \in \text{Bin}_j} (a'_{t,i}, b'_{t,i})$$

$$\text{参照光源: } (\bar{a}'_{r,j}, \bar{b}'_{r,j}) = \frac{1}{N_j} \sum_{i \in \text{Bin}_j} (a'_{r,i}, b'_{r,i}) \quad (N_j \text{ 为第 } j \text{ 个 Bin 内的色样数量})$$

(3) 多边形面积计算

分别计算 16 个平均坐标 $(\bar{a}', \bar{b}')$ 构成的多边形面积（采用鞋带公式）：

参照光源面积 S_r ：由 $(\bar{a}'_{r,1}, \bar{b}'_{r,1}), \dots, (\bar{a}'_{r,16}, \bar{b}'_{r,16})$ 构成

待测光源面积 S_t ：由 $(\bar{a}'_{t,1}, \bar{b}'_{t,1}), \dots, (\bar{a}'_{t,16}, \bar{b}'_{t,16})$ 构成

鞋带公式：对于顶点 $(a_1, b_1), \dots, (a_n, b_n)$ ，面积 S 。

$$S = 0.5 \cdot \left| \sum_{i=1}^n (a_i b_{i+1} - a_{i+1} b_i) \right| \quad (a_{n+1} = a_1, b_{n+1} = b_1) \quad (21)$$

(4) 色域指数 (Rg) 的计算结果

最终我们计算得到了色域指数 (Rg) 的值为 106.08。如下图所示，我们进一步对待测光源品质进行评估。

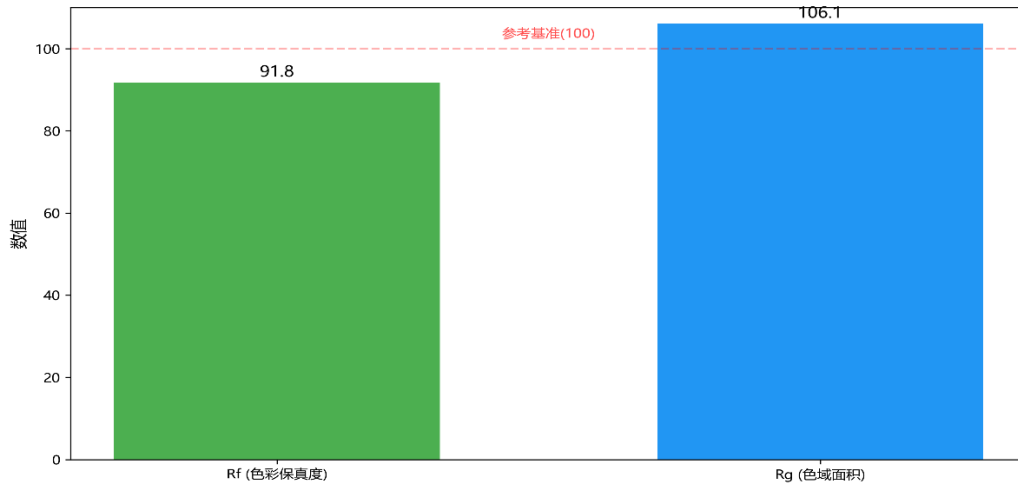


图 7 光源色品质评估指标

4.3 生理节律效应参数:褪黑素日光照度比 (mel-DER)

褪黑素日光照度比描述光源对 ipRGC（视网膜光敏神经节细胞）的刺激强度，用于衡量其影响褪黑素的能力，是光源的黑视素通量（Melanopic Flux）与相同照度下标准日光（D65）的黑视素通量的比值。

通过查阅文献，较多学者都有研究。首先，我们需要先计算 M/P 比率（黑视素与明视觉之间的比值），此指标用于评估光对人体生理节律和视觉功能的综合影响，这里简称为 MPR。

$$\text{MPR} = \frac{832 \int_{380}^{780} \Phi_e(\lambda) \cdot S_{mel}(\lambda) \cdot d\lambda}{683 \int_{380}^{780} \Phi_e(\lambda) \cdot V(\lambda) \cdot d\lambda} \quad (22)$$

其中，黑视素前置系数为 832lm/W；明视觉光谱效能最大值系数为 683lm/W
 $\Phi_e(\lambda)$ 为待测光源光谱曲线； $V(\lambda)$ 为明视觉光谱曲线； $S_{mel}(\lambda)$ 为黑视素敏感曲线（ipRGC 相应曲线）。

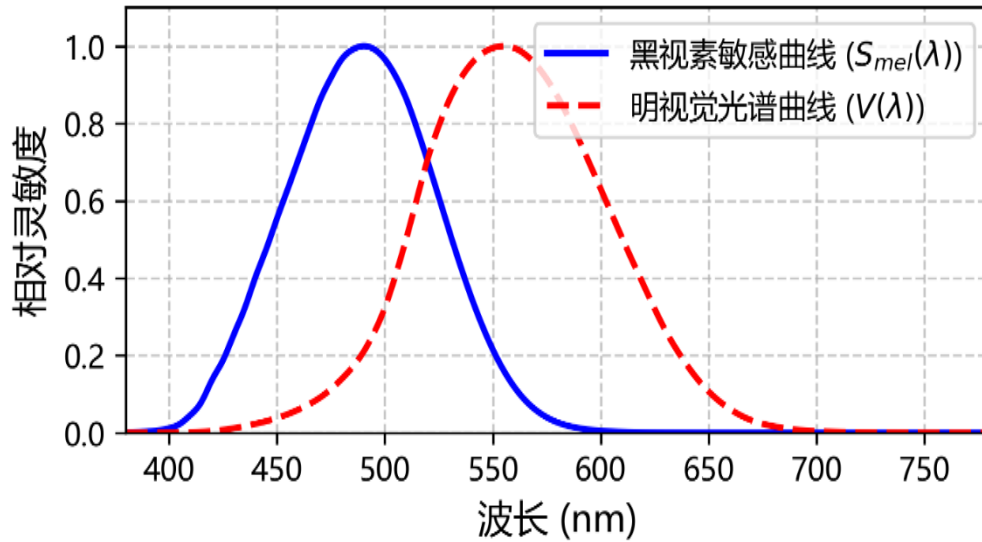


图 8 黑视素敏感曲线与明视觉光谱曲线对比图

褪黑素日光照度比与 MPR 之间存在线性关系：

$$\text{mel-DER} = \frac{1000}{1.326 \times 832} \text{MPR} \quad (23)$$

最终计算得到褪黑素日光照度比（mel-DER）的值为 0.640847。

4.4 五大特征参数值结果展示

表 1 五大参数值计算结果

类别	指标名称	简记	计算所得值
颜色特征参数	相关色温	CCT	3903.6483 K
	距离普朗克轨迹点距离	Duv	-0.001024
颜色还原参数	保真度指数	Rf	91.79
	色域指数	Rg	106.08
生理节律效应参数	褪黑素日光照度比	mel-DER	0.640847

五、问题二的模型建立与求解

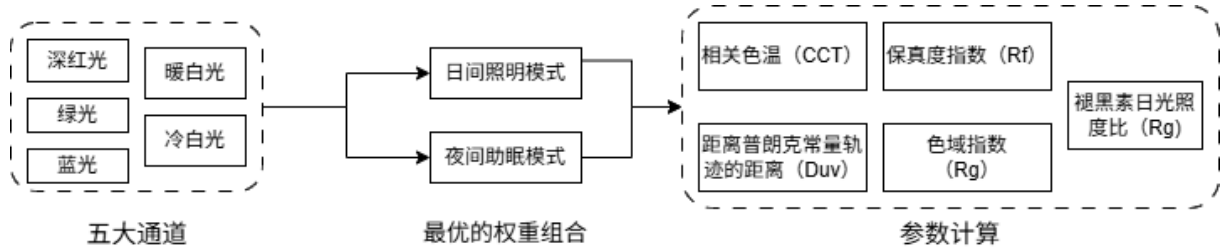


图 9 问题二的思考流程

查阅文献发现，可以采用线性叠加原理来合成任意的 LED 光谱，也就是说将多个 LED 的光谱分布进行加权求和可以得到新的光谱分布。在问题一中已经得到了 CCT、Duv、Rf、Rg、mel-DER 五个参数的求解模型和实现相关代码，这个是后续的必要准备。那么我们需要利用题目已给的五个通道，寻找最佳的权重组合来合成满足特定需求的光谱：（1）日间照明环境（2）夜间助眠模式。

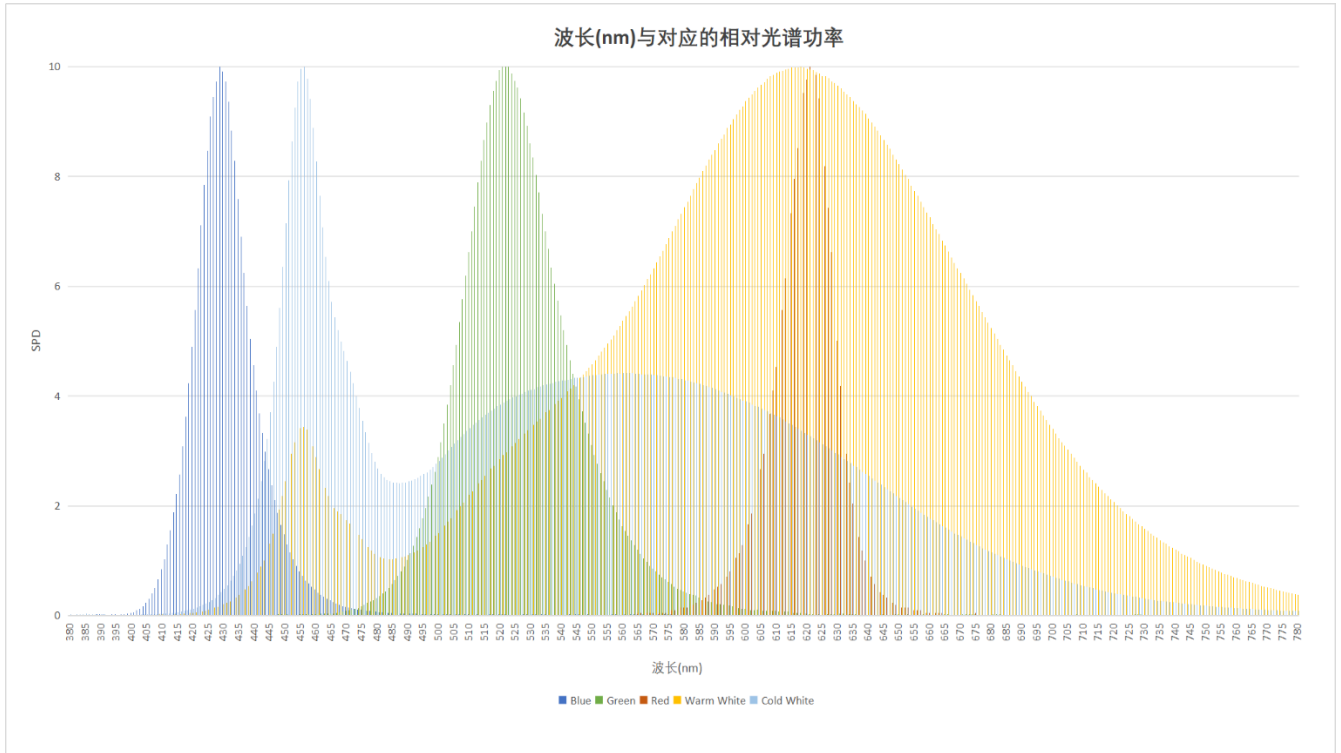


图 10 五个“通道”波长与其对应的相对光谱功率图

5.1 日间照明模式

在该模式下一共有三个条件：（1）合成光谱的保真度指数（Rf）尽可能高，接近 100；（2）合成光谱的相关色温（CCT）在正午日光范围：6000±500K 以内；（3）色域指数（Rg）在 95~105 之间，Rf>88 是可以保证颜色自然。据此，得出以下规划模型。

目标函数：

$$\max Rf \quad (24)$$

约束条件：

$$s.t \begin{cases} 5500 \leq CCT \leq 6500 \\ 95 \leq R_g \leq 105 \\ R_f > 88 \\ \alpha_{blue} + \alpha_{green} + \alpha_{red} + \alpha_{ww} + \alpha_{cw} = 1 \\ \alpha_{blue}, \alpha_{green}, \alpha_{red}, \alpha_{ww}, \alpha_{cw} \geq 0 \end{cases} \quad (25)$$

其中， α 为各通道光谱的叠加比例。

5.2 夜间助眠模式

在该模式下一共有三个条件：（1）营造控制低色温环境（相对接近度在 2500K-3500K）；（2）合成光谱的视黑素日光效率比（mel-DER）尽可能低；（3）为保证基本的颜色辨别能力，要求一般保真度指数（Rf）不低于 80。

目标函数：

$$\min \text{mel-DER} \quad (26)$$

约束条件：

$$s.t \begin{cases} 2500 \leq CCT \leq 3500 \\ R_f \geq 80 \\ \alpha_{blue} + \alpha_{green} + \alpha_{red} + \alpha_{ww} + \alpha_{cw} = 1 \\ \alpha_{blue}, \alpha_{green}, \alpha_{red}, \alpha_{ww}, \alpha_{cw} \geq 0 \end{cases} \quad (27)$$

其中， α 为各通道光谱的叠加比例。

5.3 模型求解

上述两个规划问题属于多约束非线性规划问题，我们思考从随机解的角度出发，通过迭代来寻求最优解，追随当前搜索到的最优值来寻找全局最优解。也就是采用粒子群算法进行求解，粒子群算法的流程图如下所示。

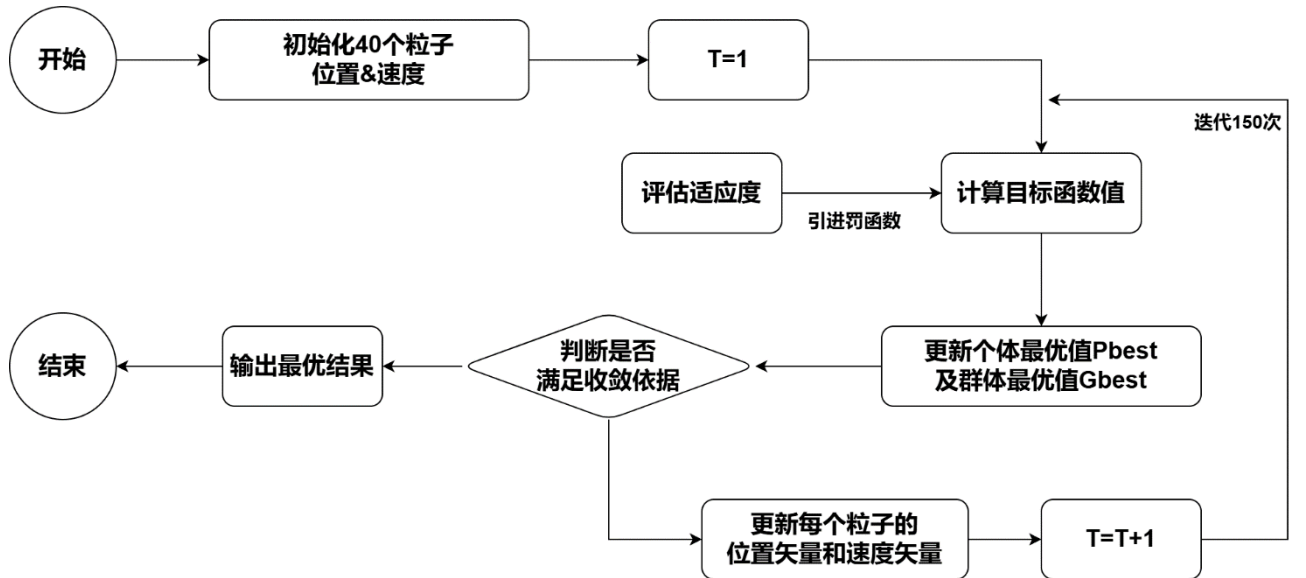


图 11 粒子群算法流程图

5.3.1 日间照明模式下的五通道最佳权重组合

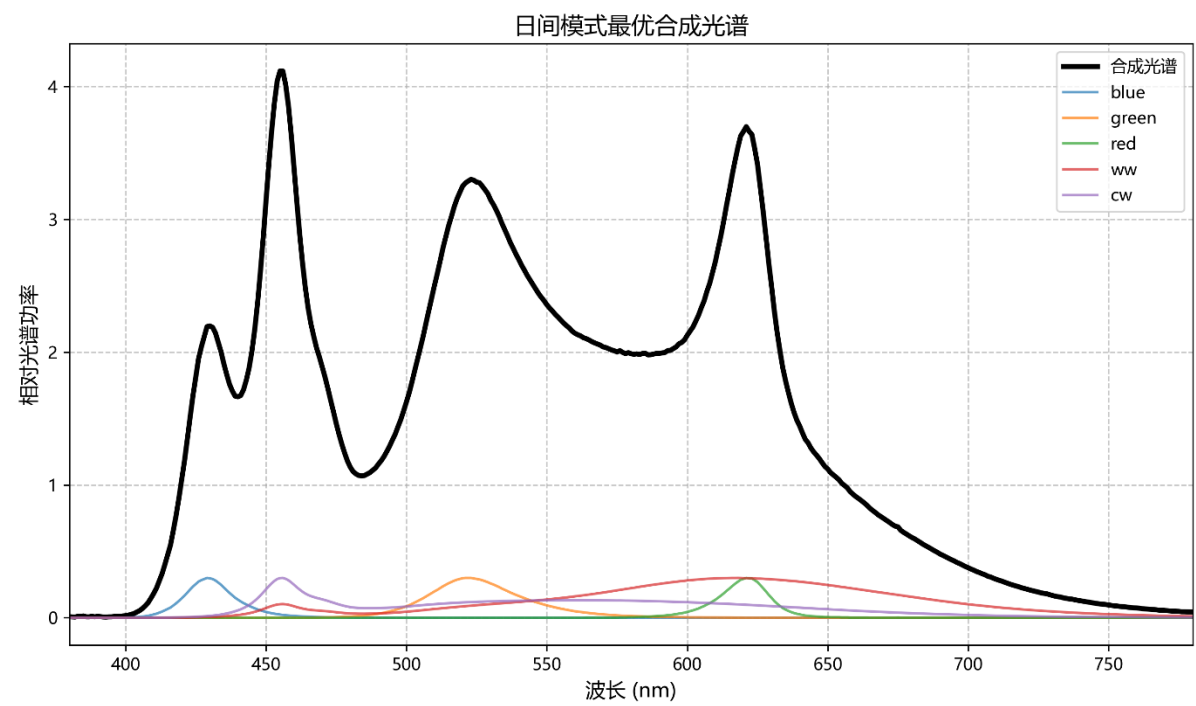


图 12 日间模式最优合成光谱

通道	深红	蓝	绿	暖白	冷白
权重	0.2335	0.1552	0.1766	0.0149	0.4199

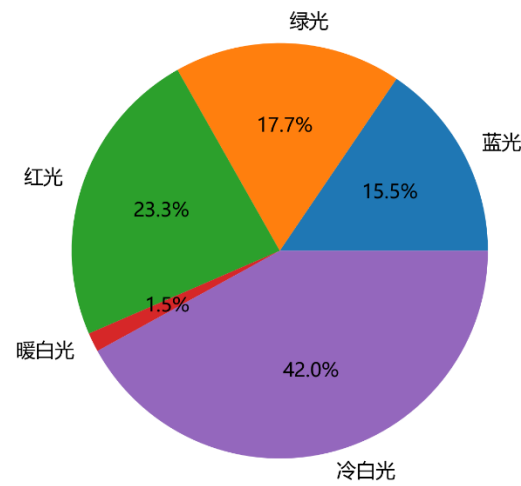


图 13 通道权重日间模式

日间照明模式的合成光谱性能参数	参数值
相关色温 (CCT)	5500.4K
距离普朗克轨迹的距离 (Duv)	0.006772
保真度指数 (Rf)	92.8
色域指数 (Rg)	102.6
褪黑素日光照度比 (mel-DEr)	0.833

5.3.2 夜间助眠模式下的五通道最佳权重组合

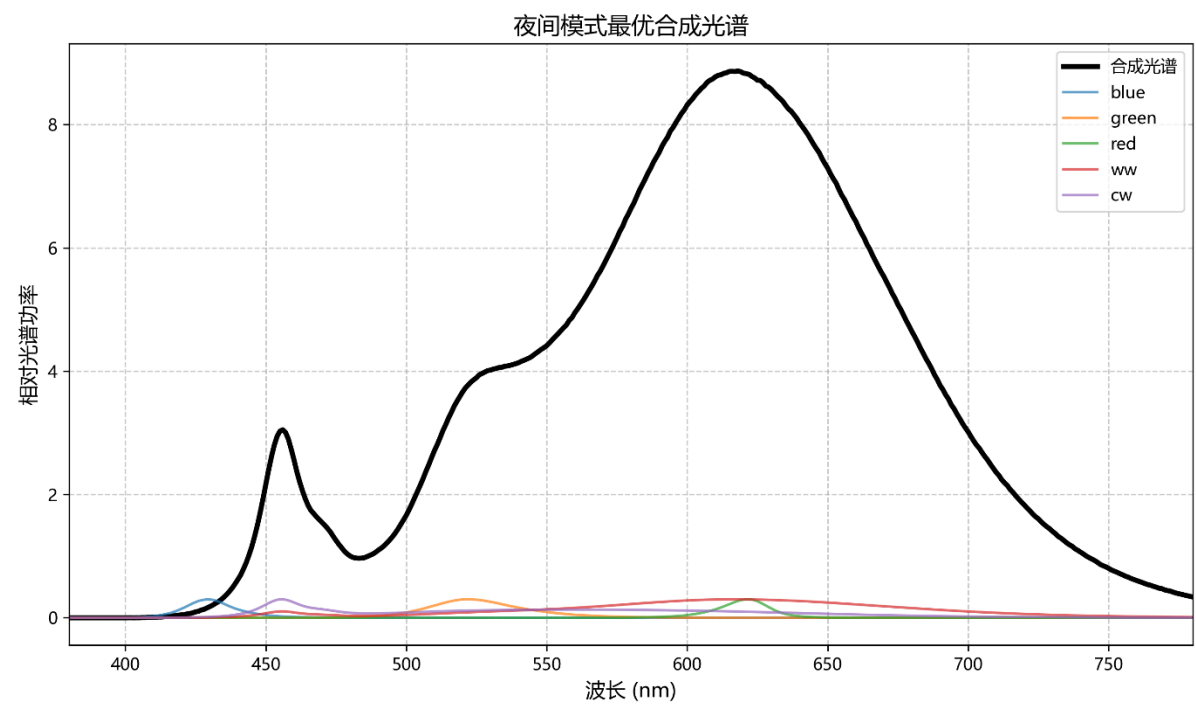


图 14 夜间模式优化过程

通道	深红	蓝	绿	暖白	冷白
权重	0.0000	0.0000	0.1136	0.8864	0.0000

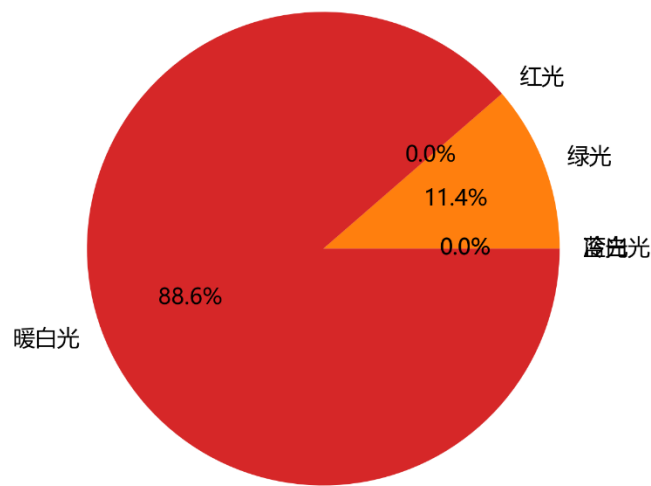


图 15 通道权重夜间模式

夜间助眠模式的合成光谱性能参数	参数值
相关色温 (CCT)	2500.0K
距离普朗克轨迹的距离 (Duv)	0.000884
保真度指数 (Rf)	88.7
色域指数 (Rg)	98.4
褪黑素日光照度比 (mel-DEr)	0.3670

5.4 模型改进

5.4.1 算法局限性

SPO 算法有着简单操作和易收敛的特点，但由于随机粒子数量，学习因子和社会因子等多种因素的存在，有陷入局部最优解的可能性，或者说找不到可行解。我们在求解的时候，也遇到了找不到可行解的情况，然后粒子被困在了这些解之中，只能重启模型。由于时间优化限制和计算机性能限制，我们仅设置 40 个随机粒子，目的是迅速找到一个可行解。

5.4.2 改进方向

我们在上面使用了罚函数法，抛弃不符合要求的粒子。但同时，我们可以动态引进新的粒子，来接替不符合要求的粒子，使得算法本身的抗风险能力增强。

本题中，学习因子和社会因子我将其设置为固定值，学习因子要稍大于社会因子，先考虑了自身的最优解迭代。其实，我们同样可以将其动态化，让目标函数的结果、适应性检验、迭代效果来反推动我们两个因子的修正。来决定扩大还是缩小最优解搜索范围，更具有准确性。

六、问题三的模型建立与求解

题目要求结合问题二中给出的五通道 LED，设计一个控制策略，使其合成的光谱能够在全天范围模拟给定的太阳光谱数据，使其具有相似的节律效果。给出了从清晨（5：30）到日落（19：30）的太阳光谱，我们要使每个时间段合成的光谱都可以模拟太阳光谱的节律效果。与问题 2 类似，仍然是寻找最佳的权重组合来合成满足特定需求的光谱。

控制策略意思就是，需要找到 LED 通道所分配的权重随时间段变化。题目要求结合问题二中给出的五通道 LED，设计一个控制策略，使其合成的光谱能够在全天范围模拟给定的太阳光谱数据，使其具有相似的节律效果。

Step1 计算太阳光谱各时间段的 CCT：

时间	CCT （单位：K）	时间	CCT （单位：K）
5:30	5338.062008	13:30	5579.694486
6:30	5314.128171	14:30	5490.264193
7:30	5289.805555	15:30	5393.695081
8:30	5265.085056	16:30	5289.125464
9:30	5375.321891	17:30	5147.3751
10:30	5477.855311	18:30	4827.714205
11:30	5573.437948	19:30	3463.706804
12:30	5662.732156		

Step2 建立模型

太阳光谱的 CCT 称为真实值，记作 T 。五个 LED 通道的 SPD 数据，是一个 401×5 的矩阵，记为 S 。本题权重组合是一个 5×15 矩阵，且向量之和均为 1，记为 W 。

S 矩阵：

$$S = \begin{bmatrix} S_1(\lambda_1) & \dots & S_5(\lambda_1) \\ \dots & \dots & \dots \\ S_1(\lambda_{401}) & \dots & S_5(\lambda_{401}) \end{bmatrix} \quad (28)$$

初始化权重矩阵:

$$W = \begin{bmatrix} \omega_1(t_1) & \dots & \omega_1(t_{15}) \\ \dots & \dots & \dots \\ \omega_5(t_1) & \dots & \omega_5(t_{15}) \end{bmatrix} \quad (29)$$

合成光谱的 CCT 矩阵 $S_{new} = S \times W$ ，是一个 401×15 矩阵。将 15 组 CCT 数据转化为 CIE 1931 标准色品坐标后，利用 McCamy 近似公式法计算 CCT，得到 15 组 CCT 数据。此时，我们得到了 15 组待求解数据和目标数据。为衡量两者间的误差，需对 15 个误差值组成的向量计算范数，以综合评估整体误差。

误差向量: $e = [e_1, e_2, \dots, e_{15}]^T$ ，其中 $e_i = |a_i - b_i|$ 是每组的绝对误差。在这里我们选用 L_∞ 范数（切比雪夫范数）：它可以输出 15 组误差中的最大值，反映“最糟糕情况”的误差。

$$\|e\|_\infty = \max |e_i| = \max \{|a_1 - b_1|, \dots, |a_{15} - b_{15}|\} \quad (30)$$

本题的目标是：最小化 15 组误差中的最大值，使所有组误差都达标。

$$\min \|CCT - T\|_\infty \quad (31)$$

与问题二类似，我们仍采用粒子群算法进行求解。最终得到如下结果。

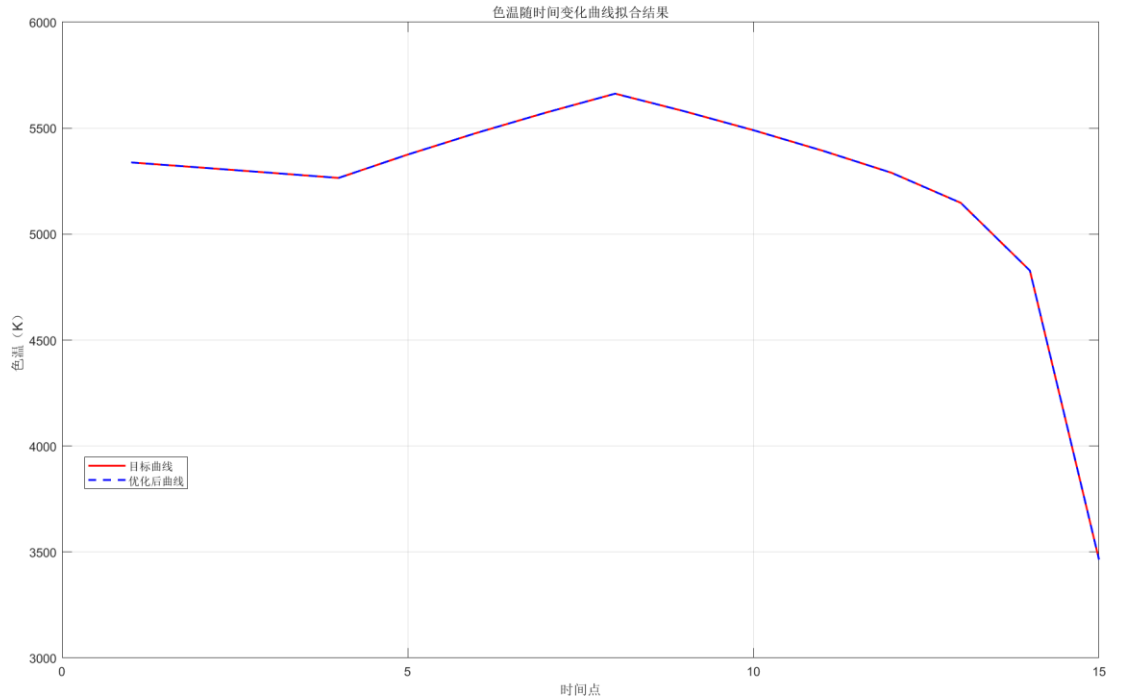


图 16 色温随时间变化曲线拟合结果

由该图看出，色温的拟合效果极好，进一步将前 8 个时间点的进行拟合，如下图所示，可以证明了我们的模型有较好的精度。

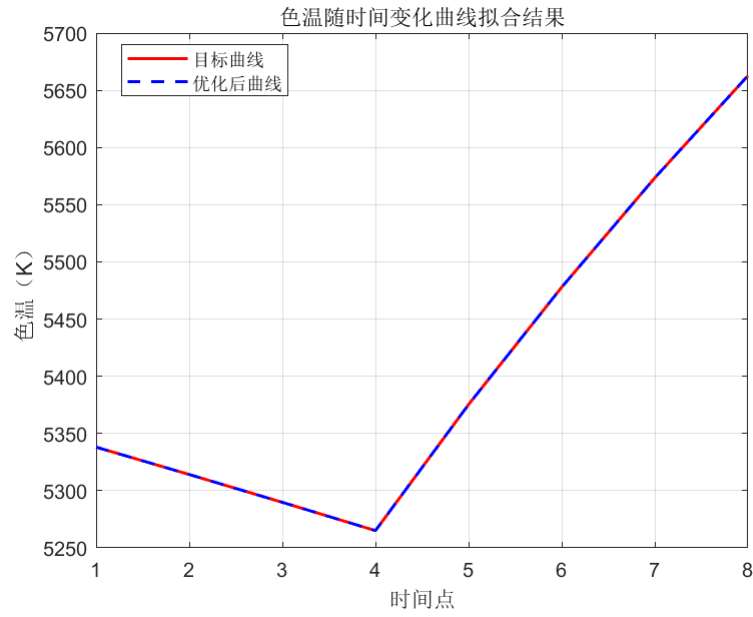


图 17 前 8 个时间点的色温拟合结果图

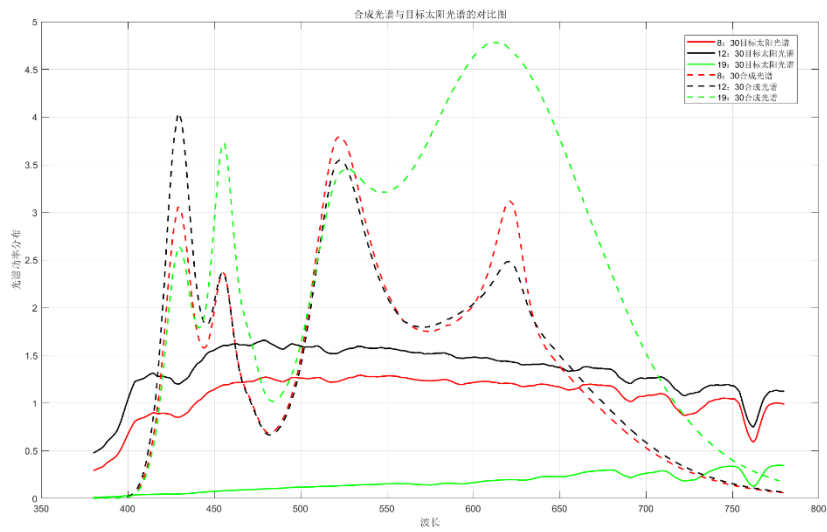


图 18 合成光谱在早、中、晚时间段的拟合效果图

由上图可知，通过上述的约束求解，将合成光谱与目标太阳光谱进行效果对比，但我们发现拟合效果不是特别优，在模型改进方面本文将思考未来的改进方向。本问的结果：控制策略，放在附件里面。

七、问题四的模型建立与求解

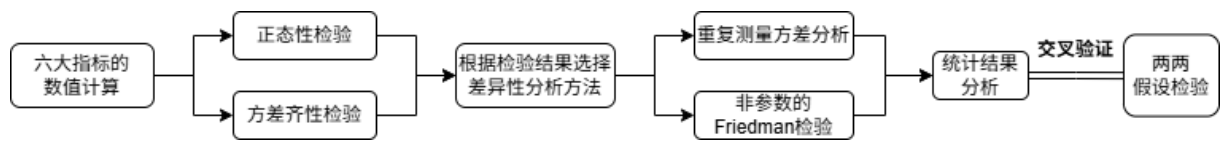


图 19 问题四的流程图

7.1 指标的计算

Step1 分析数据：题目给出的数据文件，数据结构为：行代表被试者，包含三列数据 Night1（环境 A）、Night2（环境 B）、Night3（环境 C）。每列数据是一个时间序列（30 秒间隔记录），使用数字编码代表了被试者的睡眠等级。总计 33 列。

编码	4	5	2	3
等级	清醒	REM 睡眠	N2+N1 期睡眠	N3 期睡眠

Step2 指标计算：一共有六个核心指标，如下相关性热力图所示，除了睡眠效率与总睡眠时间的相关系数达到了 0.69，以及睡眠效率和入睡潜伏期的相关系数达到-0.56，其余指标之间的相关系数都是较低的，那也证明了题目这些指标作为睡眠质量评估的合理性，后续这些指标作为综合评价模型的指标。

总睡眠时间	睡眠效率	入睡潜伏期	深睡眠比例	REM 睡眠比例	夜间醒来次数
TST	SE	SOL	N3%	REM%	

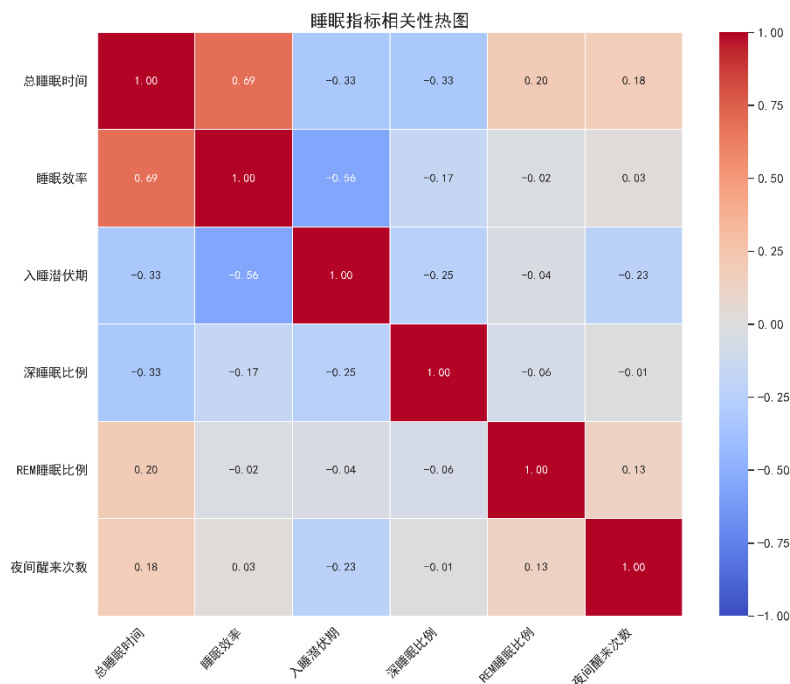


图 20 睡眠指标相关性热力图

首先确认了数据结构和计算逻辑：数据包含 11 名被试者，每个被试者有 3 个晚上的记录（Night1 环境 A、Night2 环境 B、Night3 环境 C），每个晚上以 30 秒（0.5 分钟）为一个记录点（epoch），睡眠阶段编码为 4（清醒）、5（REM）、2（浅睡眠：N1+N2）、3（深睡眠：N3）。指标计算涉及 33 个独立样本（11 人 × 3 环境）。

指标	操作（时间间隔转换为 0.5 分钟）
总睡眠时间（TST）	筛选编码：5，2，3 的单元格，求和后乘以 0.5 分钟
睡眠效率（SE）	$SE = (TST / \text{总卧床时间}) * 100\%$
入睡潜伏期（SOL）	编码由第一个 4 进入到第一个非 4（5 或 3 或 2）的时间
深睡眠比例（N3%）	$N3\% = (N3 \text{ 期总时长} / TST) * 100\%$
REM 睡眠比例（REM%）	$REM\% = (REM \text{ 期总时长} / TST) * 100\%$
夜间醒来次数	单元格编码由非 4（5 或 3 或 2）首次到 4 的次数

Step3 指标计算结果展示：

表 2 睡眠指标计算值

被试者编号	Environment	TST	SE(%)	SOL(min)	N3(%)	REM(%)	Wake_Events
1	A	310	86.95652174	15	14.67741935	20.48387097	19
1	B	312	90.04329004	2	24.51923077	23.23717949	19
1	C	288.5	82.07681366	5	25.64991334	25.64991334	19
2	A	322	65.7814096	16	17.23602484	32.14285714	17
2	B	348	75.24324324	5	30.8908046	29.74137931	13
2	C	347	91.67767503	0	27.80979827	23.63112392	16
3	A	360.5	91.84713376	4	21.3592233	26.21359223	12
3	B	260	71.33058985	51	28.07692308	20.19230769	14
3	C	259	68.24769433	54.5	21.62162162	24.13127413	12
4	A	334	83.70927318	40.5	24.7005988	33.98203593	13
4	B	386	94.37652812	13	20.20725389	42.35751295	11
4	C	347	87.40554156	7	34.14985591	21.61383285	9
5	A	397.5	93.30985915	16	26.91823899	22.51572327	16
5	B	430.5	93.99563319	16	9.407665505	36.12078978	15
5	C	384	84.3956044	2	27.21354167	18.09895833	14
6	A	382.5	89.68347011	31	14.24836601	15.29411765	11
6	B	358.5	74.60978148	75	10.87866109	21.75732218	12
6	C	365	91.70854271	2.5	26.02739726	12.46575342	11
7	A	371	84.31818182	31	18.73315364	26.68463612	14
7	B	401.5	88.33883388	47.5	20.1743462	28.01992528	8
7	C	379.5	94.28571429	7	22.66139657	7.509881423	9
8	A	381.5	89.44900352	31	12.7129751	14.02359109	13
8	B	375.5	88.45700824	2	22.63648469	20.7723036	17
8	C	435	95.39473684	2	25.97701149	35.97701149	12
9	A	491	92.29323308	6.5	19.75560081	32.58655804	22
9	B	455.5	93.43589744	13.5	20.19758507	28.9791438	25
9	C	437	95.51912568	1.5	15.90389016	20.4805492	22
10	A	356	88.33746898	7	17.41573034	32.44382022	14
10	B	453.5	96.38682253	5.5	10.80485116	26.79162073	11
10	C	347	93.91069012	9	18.1556196	24.35158501	8
11	A	329.5	77.25674091	23.5	21.39605463	21.69954476	11
11	B	389.5	96.05425401	8.5	15.53273427	17.07317073	11
11	C	367.5	87.29216152	24	26.12244898	19.31972789	20

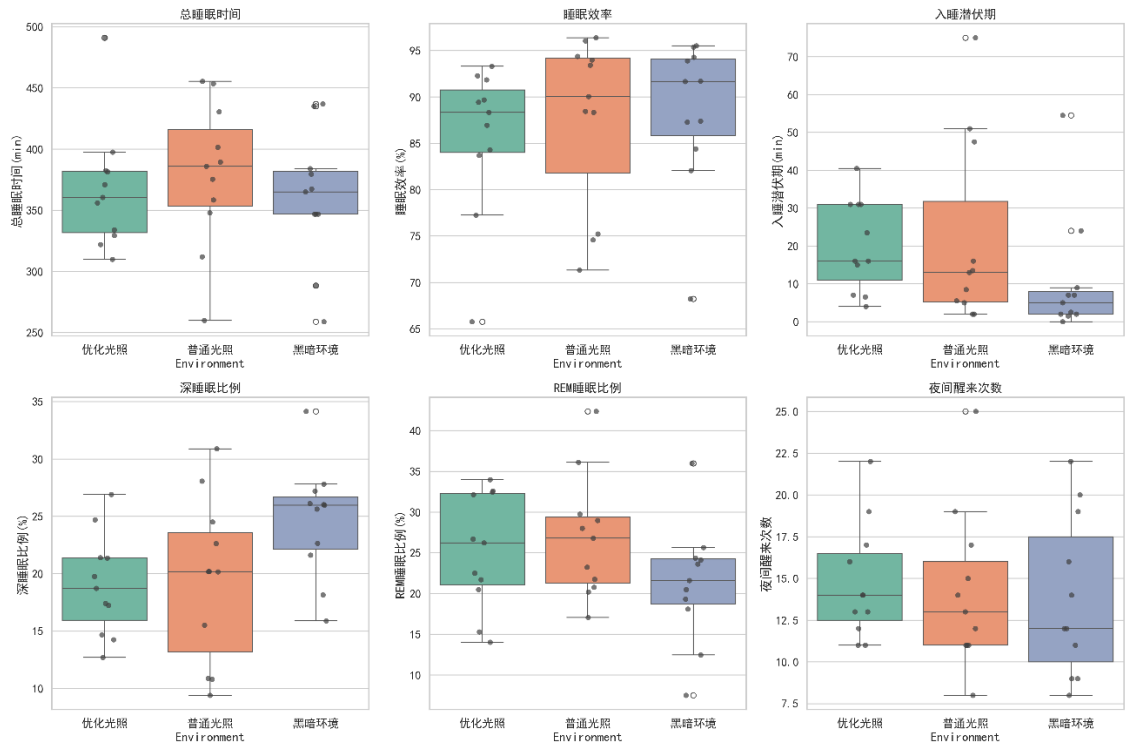


图 21 六大睡眠指标在不同环境下的效果图

7.2 指标的正态性检验与方差齐性检验

由以上已经计算出 33 个样本点的指标值，在选择差异性分析方法之前我们需要对指标进行正态性检验以及方差齐性检验，再据此结果选择合适的分析方法。

7.2.1 正态性检验

对六大指标进行正态性检验，计算结果如下图所示：

环境	指标	W	P 值	是否符合正态分布
A	总睡眠时间（TST）	0.865	0.067775228	符合正态分布
A	睡眠效率（SE）	0.825	0.020014702	不符合正态
A	入睡潜伏期（SOL）	0.928	0.390890847	符合正态分布
A	深睡眠比例（N3%）	0.969	0.872911962	符合正态分布
A	REM 睡眠比例	0.915	0.282457712	符合正态分布
A	夜间醒来次数	0.907	0.226167421	符合正态分布
B	总睡眠时间（TST）	0.955	0.704094062	符合正态分布
B	睡眠效率（SE）	0.822	0.018059531	不符合正态分布
B	入睡潜伏期（SOL）	0.780	0.005100789	不符合正态分布
B	深睡眠比例（N3%）	0.942	0.546272022	符合正态分布
B	REM 睡眠比例	0.937	0.486040263	符合正态分布
B	夜间醒来次数	0.907	0.223889958	符合正态分布
C	总睡眠时间（TST）	0.932	0.428539909	符合正态分布
C	睡眠效率（SE）	0.825	0.020198231	不符合正态分布
C	入睡潜伏期（SOL）	0.640	8.03911E-05	不符合正态分布
C	深睡眠比例（N3%）	0.942	0.543878172	符合正态分布
C	REM 睡眠比例	0.949	0.625506675	符合正态分布
C	夜间醒来次数	0.921	0.323713794	符合正态分布

7.2.2 方差齐性检验

对六大指标进行方差齐性检验，结果如下表所示：

指标	F 统计量	P 值	是否方差齐性
总睡眠时间（TST）	0.186837224	0.830536674	是
睡眠效率（SE）	0.197275496	0.822020862	是
入睡潜伏期（SOL）	0.882338771	0.424279508	是
深睡眠比例（N3%）	1.111602	0.342206307	是
REM 睡眠比例	0.079883597	0.923419514	是
夜间醒来次数	0.495732108	0.614026891	是

上述已对检验完成，若在三种环境均满足正态性，且方差齐性，则采用重复测量方差分析；若至少在一个环境不满足正态性，则采用非参数的 Friedman 检验。那么我们可以得到：（1）对于总睡眠时间、深睡眠比例、REM 睡眠比例、夜间醒来次数使用重复测量方差分析方法。（2）对于睡眠效率、入睡潜伏期这两个指标采用非参数的 Friedman 检验方法。

7.3 重复测量方差分析

对于既满足方差齐性检验又满足正态性检验的指标：总睡眠时间、深睡眠比例、REM 睡眠比例、夜间醒来次数，采用该方法判断三种不同情况的是非常合适的。

指标	F 值	分子自由度	分母自由度	P 值
总睡眠时间（TST）	0.9982	2	20	0.3862
深睡眠比例（N3%）	3.5150	2	20	0.0492
REM 睡眠比例	1.9247	2	20	0.1720
夜间醒来次数	0.3541	2	20	0.7061

深度睡眠比例的 P 值小于 0.05，因此可以得出结论在显著性水平为 95% 时，不同光照环境后对深睡眠比例有显著性影响，对总睡眠时间、REM 睡眠比例、夜间醒来次数没有显著性影响。说明仅在 N3 睡眠阶段占比这一指标上，三种光照环境存在显著差异。

7.4 非参数的 Friedman 检验

对于 SE、SOL 采用非参检验，结果如下：

指标	Friedman 检验统计量	P 值
睡眠效率（SE）	3.4545454545454675	0.17776857282813668
入睡潜伏期（SOL）	2.3333333333333325	0.31140322391459896

两个指标的 P 值都大于 0.05，因此在显著性水平为 95% 时我们可以说明不同光照环境后，对睡眠效率和入睡潜伏期都没有显著性影响，说明没有显著性差异。

7.5 结论分析

7.5.1 显著性差异结论分析

本次关于不同光照环境对睡眠指标影响的统计分析中，采用了重复测量方差分析和非参数的 Friedman 检验。对于所分析的指标，只有 N3 睡眠阶段占比这一指标对应的 P 值小于 0.05。显著性水平为 0.05 意味着当我们拒绝原假设（即认为不同光照环境对指标无显著影响）时，有不超过 5% 的可能性犯错误。因此，显著性水平为 0.05（对应 95% 的置信度）时，可以得出不同光照环境对 N3 睡眠阶段占比有显著影响的结论。而总睡眠时间（TST (min)）、REM 睡眠比例（REM (%)）、夜间醒来次数（Wake_Events）

这几个指标的 P 值均大于 0.05，说明不同光照环境对它们没有显著影响。此外，对于睡眠效率（SE (%)）和入睡潜伏期（SOL (min)）这两个指标，通过非参数的 Friedman 检验发现，不同光照环境对它们同样没有显著影响。

综上，仅在深度睡眠比例这一指标上，三种光照环境存在显著差异。并且通过描述性统计分析发现，在“优质光照”环境下的深度睡眠比例会显著低于“黑暗环境”，接下来我们将进行进一步的统计分析。

7.5.2 改善结论

对于判断“优化光照”是否对睡眠质量产生有益改善，还需进一步分析“优化光照”下 N3 睡眠阶段占比的具体数值与“普通光照”和“黑暗环境”下的对比情况。

我们使用假设检验方法，分别对“优化光照”与“普通光照”、“优化光照”，与“黑暗环境”进行 t 检验。原假设为两种光照环境下 N3 睡眠阶段占比无差异，若 p 值小于 0.05，则拒绝原假设，说明两种环境下存在显著差异。通过计算，求得优化光照与普通光照对比的 p 值为 0.8820252601303964。优化光照与黑暗光照对比的 p 值为 0.01039712764905。

“优化光照”与“普通光照”相比，对 N3 睡眠阶段占比的影响没有显著差异，只是优化光照下数据更为稳定；“优质光照”与“黑暗环境”相比，黑暗环境在提高 N3 睡眠阶段占比方面表现更优。不过，睡眠质量是一个复杂的概念，应该要综合考虑其他睡眠指标来全面评估优化光照对睡眠质量的影响。



于是我们对每个指标都进行假设检验，各个指标在不同光照环境两两对比下的 p 值，从而判断不同光照环境对各指标的影响是否存在显著差异。满足正态性的和方差齐性的，我们采用独立样本 t 检验，不满足者采用非参数的 Mann-Whitney U 检验。通过计算，我们分析各指标的 P 值结果，发现大部分指标在三种光照环境两两之间都不存在显著差异，只有入睡潜伏期（SOL(min)）指标下，优化光照与黑暗环境存在显著差异，并且进一步进行描述性统计分析。

整体来看，“优质光照”会对入睡潜伏期有正向的影响，现实意义就是“优质光照”能够缩短入睡潜伏期。仅从以上统计结果分析，难以有力证明“优化光照”能在多个方面显著提高睡眠质量。但是可以说明“优质光照”在睡眠调控方面具有较大的提升潜力。

八、模型改进

8.1 模型优点

- 1.对于问题一建立的参数计算模型,采用多方法交叉验证,可以提高结果的稳健性。
- 2.标准化计算流程,可以保证结果的可重复性。
- 3.针对日间、夜间场景设计差异化优化目标 (R_f 最大化及 $mel-DEr$ 最小化),结合粒子群算法对多约束、非线性问题的适配性,方案具有明确的工程应用价值。
- 4.问题四严格遵循统计检验流程(先正态性/方差齐性检验,再选择参数/非参数方法),两两对比检验进一步聚焦“优化光照”的效果,逻辑严密。

8.2 模型改进

- 1.引入“方法权重系数”,而非简单交叉验证。对不同学者的计算方法(如 R_f 、 $mel-DEr$ 的多种公式),根据其在权威标准(如 CIE 规范)中的认可度赋值权重,最终结果为加权融合值。
- 2.在求解上我们可以进一步调整,采用“粒子群 - 模拟退火混合算法”,在粒子群算法收敛后,通过模拟退火对最优解进行局部微调,避免陷入局部最优。同时,动态调整粒子群的惯性权重(如迭代初期大权重探索解空间,后期小权重精细寻优)。
- 3.在问题三中,我们色温进行了拟合,仍需要进行其他参数的拟合。
- 4.在第四问的统计检验中,除显著性检验外,我们进一步思考增加“效应量分析”
- 5.采用“重复测量 ANOVA+ 事后检验”,能够避免避免两两检验的 I 类错误累积。

九、参考文献

- [1] 张浩, 徐海松. 光源相关色温算法的比较研究[J]. 光学仪器, 2006(1): 54-58.
- [2] 李月. 光源相关色温及色偏差计算方法研究[D]. 辽宁科技大学, 2024.
- [3] Royer M P. Tutorial: Background and Guidance for Using the ANSI/IES TM-30 Method for Evaluating Light Source Color Rendition[J]. LEUKOS, 2022, 18(2): 191-231.

附录

(一) 附件说明:

附件文件包含:
参考资料
求解问题一的代码
求解问题二的代码
求解问题三的代码
求解问题四代码
数据文件

(二) 部分代码展示

Python 代码 问题一 (部分)

```
import numpy as np
from scipy.optimize import brentq

# 给定的色品坐标 (x, y)
x = 0.3840445003795555
y = 0.3767800565662821

# --- 黑体轨迹的 Chebyshev 法 ---
# 转换为 CIE 1960 (u, v)
denominator = -2 * x + 12 * y + 3
u_c = 4 * x / denominator
v_c = 6 * y / denominator

# 定义黑体轨迹函数
def u_bar(T):
    numerator = 0.860117757 + 1.54118254e-4 * T + 1.28641212e-7 * T**2
    denominator = 1 + 8.42420235e-4 * T + 7.08145163e-7 * T**2
    return numerator / denominator

def v_bar(T):
    numerator = 0.317398726 + 4.22806245e-5 * T + 4.20481691e-8 * T**2
    denominator = 1 - 2.89741816e-5 * T + 1.61456053e-7 * T**2
    return numerator / denominator

# 数值微分 (中心差分)
def derivative(func, T, h=0.1):
    return (func(T + h) - func(T - h)) / (2 * h)

# 目标函数
def objective(T):
```

```

    u_t = u_bar(T)
    v_t = v_bar(T)
    du_dT = derivative(u_bar, T)
    dv_dT = derivative(v_bar, T)
    return du_dT * (u_t - u_c) + dv_dT * (v_t - v_c)

# 求解相关色温 (使用布伦特法)
try:
    T_c = brentq(objective, 1000, 15000, xtol=1e-5)
    print("==黑体轨迹的 Chebyshev 法==")
    print(f"相关色温 T_c = {T_c:.4f} K")
except ValueError:
    # 如果端点符号相同, 尝试分段搜索
    T_values = np.linspace(1000, 15000, 100)
    f_values = [objective(T) for T in T_values]
    for i in range(len(T_values) - 1):
        if f_values[i] * f_values[i+1] <= 0:
            T_c = brentq(objective, T_values[i], T_values[i+1], xtol=1e-5)
            print("==黑体轨迹的 Chebyshev 法==")
            print(f"相关色温 T_c = {T_c:.4f} K")
            break

# --- McCamy 近似公式法 ---
# 计算 n
n = (x - 0.3320) / (y - 0.1858)

# 计算相关色温 T
T = -437 * n**3 + 3601 * n**2 - 6861 * n + 5514.31

print("==McCamy 近似公式法==")
print(f"相关色温 T= {T:.4f} K")

import xlrd
import numpy as np

def xy_to_uv(x, y):
    """
    将 CIE 1931 色品坐标(x,y)转换为 CIE 1960 色品坐标(u,v)
    """
    u = 4 * x / (-2 * x + 12 * y + 3)
    v = 6 * y / (-2 * x + 12 * y + 3)
    return u, v

```

```

def duv_formula_method(xt, yt):
    """
        计算 Duv
    """
    # 读取黑体曲线色品坐标数据
    data = xlrd.open_workbook('data/black_body_locus.xls')
    table = data.sheets()[0]
    m = table.nrows
    n = table.ncols
    # 将读取的数据赋给数组 black_locus
    black_locus = np.zeros((m, n))
    for i in range(m):
        for j in range(n):
            black_locus[i, j] = table.cell(i, j).value
    # 将 CIE 1931 色品坐标转换为 CIE 1960 色品坐标
    xp = black_locus[:, 1]
    yp = black_locus[:, 2]
    # 转换为 CIE 1960 色品坐标
    [up, vp] = xy_to_uv(xp, yp)
    # 将待测光源色品坐标作同样的转换
    [ut, vt] = xy_to_uv(xt, yt)
    # 用公式(1-221)求色品坐标之间的距离
    delta_c = ((ut-up)**2+(vt-vp)**2)**0.5
    # 返回距离最小值的索引
    n = np.argmin(delta_c)
    # 对 Duv 的符号进行判定
    if vt-vp[n] >= 0:
        sign_flag = 1
    else:
        sign_flag = -1
    # 计算 Duv
    duv = sign_flag*delta_c[n]
    # 返回 Duv
    return duv

def main():
    # 设置 CIE 1931 色品坐标
    xt = 0.3840445003795555
    yt = 0.3767800565662821
    duv = duv_formula_method(xt, yt)
    print('Duv =', round(duv, 6))

if __name__ == '__main__':
    main()

```

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from matplotlib.patches import Polygon
import matplotlib.colors as mcolors
from scipy.integrate import simpson
from scipy.optimize import brentq

# 设置中文字体
plt.rcParams["font.sans-serif"] = ["Microsoft YaHei"]
plt.rcParams['axes.unicode_minus'] = False # 解决负号显示问题

# 相关色温计算 - Chebyshev 法
def calculate_cct_chebyshev(x, y):
    """使用黑体轨迹的 Chebyshev 法计算相关色温"""
    # 转换为 CIE 1960 (u, v)
    denominator = -2 * x + 12 * y + 3
    u_c = 4 * x / denominator
    v_c = 6 * y / denominator

    # 黑体轨迹函数
    def u_bar(T):
        numerator = 0.860117757 + 1.54118254e-4 * T + 1.28641212e-7 * T**2
        denominator = 1 + 8.42420235e-4 * T + 7.08145163e-7 * T**2
        return numerator / denominator

    def v_bar(T):
        numerator = 0.317398726 + 4.22806245e-5 * T + 4.20481691e-8 * T**2
        denominator = 1 - 2.89741816e-5 * T + 1.61456053e-7 * T**2
        return numerator / denominator

    # 数值微分(中心差分)
    def derivative(func, T, h=0.1):
        return (func(T + h) - func(T - h)) / (2 * h)

    # 目标函数
    def objective(T):
        u_t = u_bar(T)
        v_t = v_bar(T)
        du_dT = derivative(u_bar, T)
        dv_dT = derivative(v_bar, T)
        return du_dT * (u_t - u_c) + dv_dT * (v_t - v_c)

```

```

# 求解相关色温
try:
    T_c = brentq(objective, 1000, 15000, xtol=1e-5)
    return T_c
except ValueError:
    # 分段搜索确保找到解
    T_values = np.linspace(1000, 15000, 100)
    f_values = [objective(T) for T in T_values]
    for i in range(len(T_values) - 1):
        if f_values[i] * f_values[i+1] <= 0:
            T_c = brentq(objective, T_values[i], T_values[i+1], xtol=1e-5)
            return T_c
    raise ValueError("无法计算 Chebyshev 法相关色温")

# 相关色温计算 - McCamy 近似公式法
def calculate_cct_mccamy(x, y):
    """使用 McCamy 近似公式法计算相关色温"""
    n = (x - 0.3320) / (y - 0.1858)
    T = -437 * n**3 + 3601 * n**2 - 6861 * n + 5514.31
    return T

# 数据加载函数
def load_data():
    # 读取待测光源 SPD
    spd_data = pd.read_csv('data/Problem_1.csv')
    spd_wavelengths = spd_data['wavelength'].values
    spd_values = spd_data['SPD'].values

    # 读取 10° 视角 CIE 三刺激值
    cie10_data = pd.read_csv('data/CIE_xyz_1964_10deg.csv')
    cie10_wavelengths = cie10_data['wavelength'].values
    cie10_x = cie10_data['x'].fillna(0).values
    cie10_y = cie10_data['y'].fillna(0).values
    cie10_z = cie10_data['z'].fillna(0).values

    # 读取 2° 视角 CIE 三刺激值（用于相关色温计算）
    cie2_data = pd.read_csv('data/CIE_xyz_1931_2deg.csv')
    cie2_wavelengths = cie2_data['wavelength'].values
    cie2_x = cie2_data['x'].fillna(0).values
    cie2_y = cie2_data['y'].fillna(0).values
    cie2_z = cie2_data['z'].fillna(0).values

```



```

# 读取 99 个颜色样品反射率
color_samples = pd.read_csv('data/99_color_samples.csv')
sample_wavelengths = color_samples['wavelength'].values
samples = color_samples.filter(regex='sample').values.T # 99x401 矩阵

# 读取重组日光矢量数据
daylight_data = pd.read_csv('data/common_daylight_1nm.csv')
daylight_wavelengths = daylight_data['wavelength'].values
daylight_v1 = daylight_data['V1'].values # 基础矢量 1
daylight_v2 = daylight_data['V2'].values # 基础矢量 2
daylight_v3 = daylight_data['V3'].values # 基础矢量 3

return {
    'spd': (spd_wavelengths, spd_values),
    'cie10': (cie10_wavelengths, cie10_x, cie10_y, cie10_z),
    'cie2': (cie2_wavelengths, cie2_x, cie2_y, cie2_z),
    'samples': (sample_wavelengths, samples),
    'daylight': (daylight_wavelengths, daylight_v1, daylight_v2, daylight_v3)
}

# 黑体光谱生成
def generate_blackbody_spectrum(tc, wavelengths):
    h = 6.62607015e-34
    c = 299792458
    k = 1.380649e-23
    a = 2 * np.pi * h * c **2
    b = h * c / k

    p_lambda = []
    for wl in wavelengths:
        lambda_m = wl * 1e-9 # 转换为米
        exponent = b / (lambda_m * tc)
        if exponent > 700: # 避免指数溢出
            p = 0.0
        else:
            p = a * (lambda_m**-5) / (np.exp(exponent) - 1)
        p_lambda.append(p)

    # 归一化
    max_p = max(p_lambda) if p_lambda else 1.0
    return np.array(p_lambda) / max_p

# 重组日光光谱生成
def composed_daylight(tc, wavelengths, daylight_v1, daylight_v2, daylight_v3):

```

```

"""根据重组日光矢量计算光谱"""
# 计算重组日光的色度坐标 xD, yD
if tc > 25000:
    xd = 0.250
elif tc > 7000:
    xd = -2.0064e9 / (tc ** 3) + 1.9018e6 / (tc ** 2) + 0.24748e3 / tc +
0.237040
    elif tc >= 4000:
        xd = -4.6070e9 / (tc ** 3) + 2.9678e6 / (tc ** 2) + 0.09911e3 / tc +
0.244063
    else:
        xd = 0.380 # 低于 4000K 时的默认值
yd = -3.000 * (xd ** 2) + 2.870 * xd - 0.275 # yD 与 xD 的关系公式

# 计算矢量因子 M1 和 M2
denominator = 0.0241 + 0.2562 * xd - 0.7341 * yd
if abs(denominator) < 1e-9: # 避免分母为 0
    m1, m2 = 0.0, 0.0
else:
    m1 = (-1.3515 - 1.7703 * xd + 5.9114 * yd) / denominator
    m2 = (0.0300 - 31.4424 * xd + 30.0717 * yd) / denominator

# 生成重组日光光谱
spectrum = daylight_v1 + m1 * daylight_v2 + m2 * daylight_v3

# 确保光谱非负
spectrum[spectrum < 0] = 0.0
return spectrum

# 混合光谱生成
def mix_blackbody_daylight(tc, wavelengths, cie2_x, cie2_y, cie2_z, daylight_v1,
daylight_v2, daylight_v3):
    # 生成黑体和重组日光光谱
    srp = generate_blackbody_spectrum(tc, wavelengths)
    srd = composed_daylight(tc, wavelengths, daylight_v1, daylight_v2,
daylight_v3)

    # 按 2° 视角明度函数归一化
    sum_y_srp = simpson(srp * cie2_y, wavelengths)
    srp = 100 * srp / sum_y_srp if sum_y_srp != 0 else srp
    sum_y_srd = simpson(srd * cie2_y, wavelengths)
    srd = 100 * srd / sum_y_srd if sum_y_srd != 0 else srd

    # 线性混合

```

```

    tb, te = 4000, 5000
    return (tc - tb)/(te - tb) * srd + (te - tc)/(te - tb) * srp

# 改进的相关色温计算函数
def correlated_color_temperature(spectrum, wavelengths, cie2_x, cie2_y, cie2_z,
method='chebyshev'):
    """
    计算相关色温，支持两种方法

    参数：
        spectrum: 光源光谱
        wavelengths: 波长数组
        cie2_x, cie2_y, cie2_z: 2° 视角三刺激值
        method: 计算方法, 'chebyshev'或'mccamy'

    返回：
        相关色温值(K)
    """
    # 计算三刺激值
    sum_x = simpson(spectrum * cie2_x, wavelengths)
    sum_y = simpson(spectrum * cie2_y, wavelengths)
    sum_z = simpson(spectrum * cie2_z, wavelengths)

    # 计算色品坐标
    sum_xyz = sum_x + sum_y + sum_z
    if sum_xyz == 0:
        raise ValueError("无法计算三刺激值总和为零的光谱色温")

    x = sum_x / sum_xyz
    y = sum_y / sum_xyz

    # 选择计算方法
    if method == 'chebyshev':
        return calculate_cct_chebyshev(x, y)
    elif method == 'mccamy':
        return calculate_cct_mccamy(x, y)
    else:
        raise ValueError("不支持的计算方法，可选'chebyshev'或'mccamy'")

# 光源三刺激值计算
def light_source_stimulus(spectrum, wavelengths, cie_x, cie_y, cie_z):
    sum_y = simpson(spectrum * cie_y, wavelengths)
    k = 100 / sum_y if sum_y != 0 else 1.0
    x = k * simpson(spectrum * cie_x, wavelengths)

```

```

    z = k * simpson(spectrum * cie_z, wavelengths)
    return {'x_capital': x, 'y_capital': 100.0, 'z_capital': z, 'k': k}

# 色品坐标计算
def get_chromaticity(x, y, z):
    sum_xyz = x + y + z
    return (x/sum_xyz, y/sum_xyz) if sum_xyz != 0 else (0.0, 0.0)

# 样品三刺激值计算
def color_sample_stimulus(sample, spectrum, wavelengths, k, cie_x, cie_y,
cie_z):
    x = k * simpson(sample * spectrum * cie_x, wavelengths)
    y = k * simpson(sample * spectrum * cie_y, wavelengths)
    z = k * simpson(sample * spectrum * cie_z, wavelengths)
    return x, y, z

# 色适应转换
def von_kries_adapt(r, g, b, rw, gw, bw):
    rc = 100 * r / rw if rw != 0 else 0
    gc = 100 * g / gw if gw != 0 else 0
    bc = 100 * b / bw if bw != 0 else 0
    return rc, gc, bc

# XYZ 转 RGB
def xyz_to_rgb(x, y, z):
    mcat02 = [
        [0.7328, 0.4296, -0.1624],
        [-0.7036, 1.6975, 0.0061],
        [0.0030, 0.0136, 0.9834]
    ]
    return (
        mcat02[0][0]*x + mcat02[0][1]*y + mcat02[0][2]*z,
        mcat02[1][0]*x + mcat02[1][1]*y + mcat02[1][2]*z,
        mcat02[2][0]*x + mcat02[2][1]*y + mcat02[2][2]*z
    )

# RGB 转 XYZ
def rgb_to_xyz(r, g, b):
    m1cat02 = [
        [1.096124, -0.278869, 0.182745],
        [0.454369, 0.473533, 0.072098],
        [-0.009628, -0.005698, 1.015326]
    ]
    return (

```

```

        m1cat02[0][0]*r + m1cat02[0][1]*g + m1cat02[0][2]*b,
        m1cat02[1][0]*r + m1cat02[1][1]*g + m1cat02[1][2]*b,
        m1cat02[2][0]*r + m1cat02[2][1]*g + m1cat02[2][2]*b
    )

# Hunt-Pointer-Estevez 变换
def hunt_pointer_estevez(x, y, z):
    hpe = [
        [0.38971, 0.68898, -0.07868],
        [-0.22981, 1.18340, 0.04641],
        [0, 0, 1]
    ]
    return (
        hpe[0][0]*x + hpe[0][1]*y + hpe[0][2]*z,
        hpe[1][0]*x + hpe[1][1]*y + hpe[1][2]*z,
        hpe[2][0]*x + hpe[2][1]*y + hpe[2][2]*z
    )

# 亮度适应
def luminance_adaptation(rp, gp, bp):
    f1 = 0.7937
    def adapt(v):
        val = (f1 * v / 100) **0.42
        return 400 * val / (27.13 + val) + 0.1
    return adapt(rp), adapt(gp), adapt(bp)

# ab 通道计算
def channel_ab(r, g, b):
    aw = r - 12/11 * g + 1/11 * b
    bw = 1/9 * (r + g - 2*b)
    aW = 1.0003 * (2*r + g + 1/20*b - 0.305)
    return aw, bw, aW

# 色调角分箱
def count_hue_angle(hue_angles):
    return [int(angle / 22.5) + 1 for angle in hue_angles]

# 多边形面积计算
def polygon_area(a, b):
    if len(a) != len(b):
        raise ValueError("数组长度必须相同")
    n = len(a)
    a = np.append(a, a[0])
    b = np.append(b, b[0])

```

```

    area = 0.0
    for i in range(n):
        area += a[i] * b[i+1] - a[i+1] * b[i]
    return 0.5 * abs(area)

# 主计算函数
def calculate_rf_rg_with_visual_data():
    data = load_data()
    spd_wavelengths, spd_values = data['spd']
    cie10_wl, cie10_x, cie10_y, cie10_z = data['cie10']
    cie2_wl, cie2_x, cie2_y, cie2_z = data['cie2']
    sample_wl, samples = data['samples']
    daylight_wl, daylight_v1, daylight_v2, daylight_v3 = data['daylight']

    # 波长一致性检查
    if not (np.array_equal(spd_wavelengths, cie10_wl) and
            np.array_equal(spd_wavelengths, cie2_wl) and
            np.array_equal(spd_wavelengths, sample_wl) and
            np.array_equal(spd_wavelengths, daylight_wl)):
        raise ValueError("所有数据的波长必须一致")

    # 待测光谱归一化
    max_spd = max(spd_values) if spd_values.size else 1.0
    testing_spectrum = spd_values / max_spd

    # 计算相关色温（使用 Chebyshev 法，更精确）
    tc_chebyshev = correlated_color_temperature(
        testing_spectrum, spd_wavelengths, cie2_x, cie2_y, cie2_z,
        method='chebyshev'
    )
    tc_mccamy = correlated_color_temperature(
        testing_spectrum, spd_wavelengths, cie2_x, cie2_y, cie2_z,
        method='mccamy'
    )
    print(f"Chebyshev 法计算的相关色温: {tc_chebyshev:.0f} K")
    print(f"McCamy 法计算的相关色温: {tc_mccamy:.0f} K")
    # 使用更精确的 Chebyshev 法结果
    tc = tc_chebyshev

    # 根据色温选择参照光谱
    if tc < 4000:
        reference_spectrum = generate_blackbody_spectrum(tc, spd_wavelengths)
    elif tc > 5000:

```

```

        reference_spectrum = composed_daylight(tc, spd_wavelengths, daylight_v1,
daylight_v2, daylight_v3)
    else:
        reference_spectrum = mix_blackbody_daylight(
            tc, spd_wavelengths, cie2_x, cie2_y, cie2_z,
            daylight_v1, daylight_v2, daylight_v3
        )

    # 后续计算逻辑
    test_stim = light_source_stimulus(testing_spectrum, spd_wavelengths,
cie10_x, cie10_y, cie10_z)
    ref_stim = light_source_stimulus(reference_spectrum, spd_wavelengths,
cie10_x, cie10_y, cie10_z)

    test_x, test_y = get_chromaticity(test_stim['x_capital'],
test_stim['y_capital'], test_stim['z_capital'])
    ref_x, ref_y = get_chromaticity(ref_stim['x_capital'],
ref_stim['y_capital'], ref_stim['z_capital'])

    sample_count = 99
    test_samples = []
    ref_samples = []
    for i in range(sample_count):
        sample = samples[i] if i < len(samples) else np.zeros_like(spd_values)
        tx, ty, tz = color_sample_stimulus(sample, testing_spectrum,
spd_wavelengths, test_stim['k'], cie10_x, cie10_y, cie10_z)
        rx, ry, rz = color_sample_stimulus(sample, reference_spectrum,
spd_wavelengths, ref_stim['k'], cie10_x, cie10_y, cie10_z)
        test_samples.append((tx, ty, tz))
        ref_samples.append((rx, ry, rz))

    test_rgb = xyz_to_rgb(test_stim['x_capital'], test_stim['y_capital'],
test_stim['z_capital'])
    ref_rgb = xyz_to_rgb(ref_stim['x_capital'], ref_stim['y_capital'],
ref_stim['z_capital'])

    test_rc, test_gc, test_bc = von_kries_adapt(*test_rgb, *test_rgb)
    ref_rc, ref_gc, ref_bc = von_kries_adapt(*ref_rgb, *ref_rgb)

    test_xct, test_yct, test_zct = rgb_to_xyz(test_rc, test_gc, test_bc)
    ref_xcr, ref_ycr, ref_zcr = rgb_to_xyz(ref_rc, ref_gc, ref_bc)

    test_rp, test_gp, test_bp = hunt_pointer_estevez(test_xct, test_yct,
test_zct)

```

```

ref_rp, ref_gp, ref_bp = hunt_pointer_estevez(ref_xcr, ref_ycr, ref_zcr)

test_ra, test_ga, test_ba = luminance_adaptation(test_rp, test_gp, test_bp)
ref_ra, ref_ga, ref_ba = luminance_adaptation(ref_rp, ref_gp, ref_bp)

jt, ht, mt = [], [], []
jr, hr, mr = [], [], []
for i in range(sample_count):
    tx, ty, tz = test_samples[i]
    tr, tg, tb = xyz_to_rgb(tx, ty, tz)
    trc, tgc, tbc = von_kries_adapt(tr, tg, tb, *test_rgb)
    trp, tgp, tbp = rgb_to_xyz(trc, tgc, tbc)
    trp, tgp, tbp = hunt_pointer_estevez(trp, tgp, tbp)
    tra, tga, tba = luminance_adaptation(trp, tgp, tbp)
    taw, tbw, ta = channel_ab(tra, tga, tba)

    rx, ry, rz = ref_samples[i]
    rr, rg, rb = xyz_to_rgb(rx, ry, rz)
    rrc, rgc, rbc = von_kries_adapt(rr, rg, rb, *ref_rgb)
    rrp, rgp, rbp = rgb_to_xyz(rrc, rgc, rbc)
    rrp, rgp, rbp = hunt_pointer_estevez(rrp, rgp, rbp)
    rra, rga, rba = luminance_adaptation(rrp, rgp, rbp)
    raw, rbw, ra = channel_ab(rra, rga, rba)

    at = channel_ab(test_ra, test_ga, test_ba)[2]
    ar = channel_ab(ref_ra, ref_ga, ref_ba)[2]
    jt_val = 100 * (ta / at)**(0.69 * 1.9272) if at else 0.0
    jr_val = 100 * (ra / ar)**(0.69 * 1.9272) if ar else 0.0

    h_angle_t = np.degrees(np.arctan2(tbw, taw)) if (taw or tbw) else 0.0
    h_angle_t = h_angle_t if h_angle_t >= 0 else h_angle_t + 360
    h_angle_r = np.degrees(np.arctan2(rbw, raw)) if (raw or rbw) else 0.0
    h_angle_r = h_angle_r if h_angle_r >= 0 else h_angle_r + 360

    et_t = 0.25 * (np.cos(np.radians(h_angle_t)) + 2) + 3.8
    et_r = 0.25 * (np.cos(np.radians(h_angle_r)) + 2) + 3.8

    den_t = tra + tga + 21/20 * tba
    t_t = (50000/13 * 1.0003 * et_t * np.hypot(taw, tbw)) / den_t if den_t
else 0.0

    den_r = rra + rga + 21/20 * rba
    t_r = (50000/13 * 1.0003 * et_r * np.hypot(raw, rbw)) / den_r if den_r
else 0.0

```



```

        c_t = t_t**0.9 * np.sqrt(jt_val / 100) * (1.64 - 0.29**0.2)** 0.73 if
jt_val >= 0 else 0.0
        c_r = t_r**0.9 * np.sqrt(jr_val / 100) * (1.64 - 0.29**0.2)** 0.73 if
jr_val >= 0 else 0.0

        mt_val = c_t * 0.7937**0.25
        mr_val = c_r * 0.7937**0.25

        jt.append(jt_val)
        ht.append(h_angle_t)
        mt.append(mt_val)
        jr.append(jr_val)
        hr.append(h_angle_r)
        mr.append(mr_val)

        jpt_uc = [(1 + 100*0.007)*j/(1 + 0.007*j) if j else 0 for j in jt]
        jpr_uc = [(1 + 100*0.007)*j/(1 + 0.007*j) if j else 0 for j in jr]
        mpt_uc = [np.log(1 + 0.0228*m)/0.0228 if m >=0 else 0 for m in mt]
        mpr_uc = [np.log(1 + 0.0228*m)/0.0228 if m >=0 else 0 for m in mr]

        apt_uc = [mpt_uc[i] * np.cos(np.radians(ht[i])) for i in
range(sample_count)]
        apr_uc = [mpr_uc[i] * np.cos(np.radians(hr[i])) for i in
range(sample_count)]
        bpt_uc = [mpt_uc[i] * np.sin(np.radians(ht[i])) for i in
range(sample_count)]
        bpr_uc = [mpr_uc[i] * np.sin(np.radians(hr[i])) for i in
range(sample_count)]

        de = [round(np.sqrt((jpt_uc[i]-jpr_uc[i])**2 + (apt_uc[i]-apr_uc[i])** 2 +
(bpt_uc[i]-bpr_uc[i])**2), 2)
        for i in range(sample_count)]
        de_ave = sum(de)/sample_count if sample_count else 0.0

        rfi = [round(10 * np.log(np.exp((100 - 6.73*d)/10) + 1)) for d in de]
        rf = 10 * np.log(np.exp((100 - 6.73*de_ave)/10) + 1) if de_ave else 0.0

        bin_number = count_hue_angle(hr)
        jptj = np.zeros(16)
        aptj = np.zeros(16)
        bptj = np.zeros(16)
        jprj = np.zeros(16)
        aprj = np.zeros(16)

```

```

bprj = np.zeros(16)
per_bin_count = np.zeros(16, dtype=int)

for i in range(sample_count):
    bin_idx = bin_number[i] - 1
    if 0 <= bin_idx < 16:
        per_bin_count[bin_idx] += 1
        jptj[bin_idx] += jpt_uc[i]
        aptj[bin_idx] += apt_uc[i]
        bptj[bin_idx] += bpt_uc[i]
        jprj[bin_idx] += jpr_uc[i]
        aprj[bin_idx] += apr_uc[i]
        bprj[bin_idx] += bpr_uc[i]

for i in range(16):
    if per_bin_count[i] > 0:
        jptj[i] /= per_bin_count[i]
        aptj[i] /= per_bin_count[i]
        bptj[i] /= per_bin_count[i]
        jprj[i] /= per_bin_count[i]
        aprj[i] /= per_bin_count[i]
        bprj[i] /= per_bin_count[i]

valid_bins = [i for i in range(16) if per_bin_count[i] > 0]
area_test = polygon_area([aptj[i] for i in valid_bins], [bptj[i] for i in
valid_bins])
area_ref = polygon_area([aprj[i] for i in valid_bins], [bprj[i] for i in
valid_bins])
rg = 100 * area_test / area_ref if area_ref else 0.0

return {
    'Rf': round(rf, 2),
    'Rg': round(rg, 2),
    'de_values': de,
    'test_a': apt_uc,
    'test_b': bpt_uc,
    'ref_a': apr_uc,
    'ref_b': bpr_uc,
    'hue_distribution': per_bin_count,
    'rfi_values': rfi,
    'cct_chebyshev': round(tc_chebyshev, 2),
    'cct_mccamy': round(tc_mccamy, 2)
}

```

```

# 可视化函数
def plot_rf_rg(rf, rg):
    fig, ax = plt.subplots(figsize=(10, 6))
    metrics = ['Rf (色彩保真度)', 'Rg (色域面积)']
    values = [rf, rg]
    bars = ax.bar(metrics, values, color=['#4CAF50', '#2196F3'], width=0.6)

    for bar in bars:
        height = bar.get_height()
        ax.text(bar.get_x() + bar.get_width()/2., height + 0.5,
                f'{height:.1f}', ha='center', va='bottom', fontsize=12)

    ax.set_title('光源色品质评估指标', fontsize=16, pad=20)
    ax.set_ylim(0, 110)
    ax.set_ylabel('数值', fontsize=12)
    ax.axhline(y=100, color='r', linestyle='--', alpha=0.3)
    ax.text(0.5, 102, '参考基准(100)', ha='center', va='bottom', color='r',
    alpha=0.7)
    plt.tight_layout()
    return fig

def plot_de_distribution(de_values):
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))
    sns.histplot(de_values, kde=True, ax=ax1, color='#FF9800', bins=15)
    ax1.set_title('色貌差(DE)分布', fontsize=14)
    ax1.set_xlabel('色貌差(DE)', fontsize=12)
    ax1.set_ylabel('频数', fontsize=12)
    ax1.axvline(np.mean(de_values), color='r', linestyle='--', label=f'均值:
{np.mean(de_values):.2f}')
    ax1.legend()

    sns.boxplot(y=de_values, ax=ax2, color='#FF9800')
    ax2.set_title('色貌差(DE)箱线图', fontsize=14)
    ax2.set_ylabel('色貌差(DE)', fontsize=12)
    ax2.text(0.1, np.mean(de_values), f'均值: {np.mean(de_values):.2f}',
            ha='left', va='center', color='r')
    plt.tight_layout()
    return fig

def plot_hue_comparison(test_a, test_b, ref_a, ref_b, hue_distribution):
    fig, ax = plt.subplots(figsize=(10, 10))
    test_points = np.column_stack((test_a, test_b))
    ref_points = np.column_stack((ref_a, ref_b))

```

```

    test_angles = np.arctan2(test_points[:,1], test_points[:,0])
    test_sorted = test_points[np.argsort(test_angles)]
    test_poly = Polygon(test_sorted, fill=True, alpha=0.4, color='#2196F3',
label='待测光源')

    ref_angles = np.arctan2(ref_points[:,1], ref_points[:,0])
    ref_sorted = ref_points[np.argsort(ref_angles)]
    ref_poly = Polygon(ref_sorted, fill=True, alpha=0.4, color='#4CAF50',
label='参考光源')

    ax.add_patch(test_poly)
    ax.add_patch(ref_poly)
    ax.scatter(test_points[:,0], test_points[:,1], color='#2196F3', s=30,
alpha=0.7)
    ax.scatter(ref_points[:,0], ref_points[:,1], color='#4CAF50', s=30,
alpha=0.7)

    ax.set_title('光源色调范围比较', fontsize=16, pad=20)
    ax.set_xlabel('a* (红绿色轴)', fontsize=12)
    ax.set_ylabel('b* (黄蓝色轴)', fontsize=12)
    ax.axis('equal')
    ax.grid(True, linestyle='--', alpha=0.7)
    ax.legend()
    plt.tight_layout()
    return fig

def plot_rf_vs_de(de_values, rfi_values):
    fig, ax = plt.subplots(figsize=(10, 6))
    scatter = ax.scatter(de_values, rfi_values, c=de_values, cmap='viridis',
                        alpha=0.7, edgecolors='w', s=50)
    cbar = plt.colorbar(scatter)
    cbar.set_label('色貌差(DE)', fontsize=12)

    z = np.polyfit(de_values, rfi_values, 3)
    p = np.poly1d(z)
    sorted_de = np.sort(de_values)
    ax.plot(sorted_de, p(sorted_de), "r--")

    ax.set_title('色貌差(DE)与 Rf 贡献值的关系', fontsize=16, pad=20)
    ax.set_xlabel('色貌差(DE)', fontsize=12)
    ax.set_ylabel('Rf 贡献值', fontsize=12)
    ax.grid(True, linestyle='--', alpha=0.7)
    plt.tight_layout()
    return fig

```

```

# 主函数
def main():
    result = calculate_rf_rg_with_visual_data()
    print(f"计算结果: Rf={result['Rf']}, Rg={result['Rg']}")

    # 生成并保存图表
    fig1 = plot_rf_rg(result['Rf'], result['Rg'])
    fig2 = plot_de_distribution(result['de_values'])
    fig3 = plot_hue_comparison(
        result['test_a'], result['test_b'],
        result['ref_a'], result['ref_b'],
        result['hue_distribution']
    )
    fig4 = plot_rf_vs_de(result['de_values'], result['rfi_values'])

    fig1.savefig('rf_rg_metrics.png', dpi=300, bbox_inches='tight')
    fig2.savefig('de_distribution.png', dpi=300, bbox_inches='tight')
    fig3.savefig('hue_comparison.png', dpi=300, bbox_inches='tight')
    fig4.savefig('rf_vs_de.png', dpi=300, bbox_inches='tight')
    plt.show()

if __name__ == "__main__":
    main()

```